



CPE 334

SEMESTER 1/2569 · WEEK 7 · PROJECT EXECUTION

Project execution

From the plan to the increment

INSTRUCTOR

Dr. Piyanit E.

Dr. Suthep M.

Dr. Santawat T.

DEPARTMENT

Computer Engineering, KMUTT

 PART 01

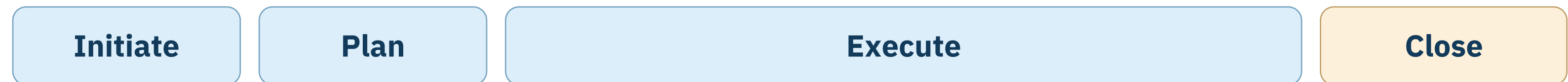
Execution in the life cycle

Where the code actually gets written

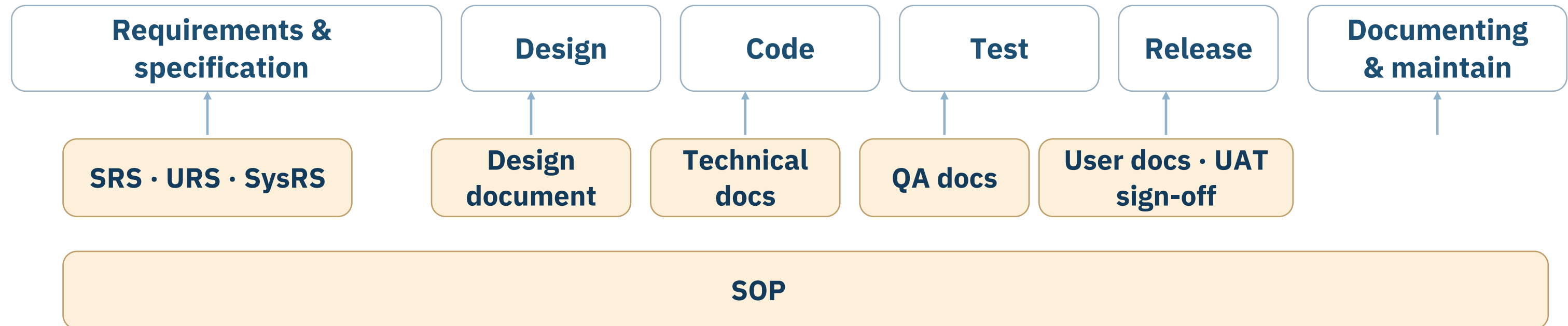
— TWO LIFE CYCLES

Project life cycle vs SDLC

PROJECT LIFE CYCLE

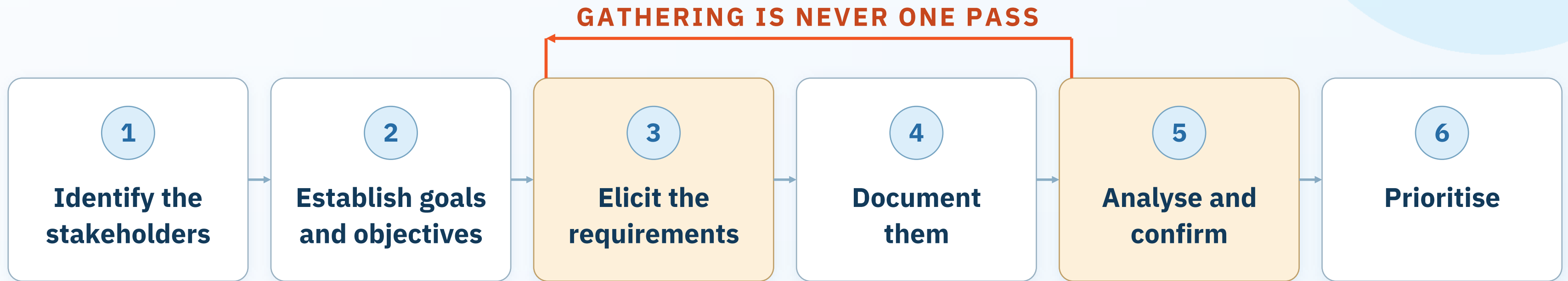


SOFTWARE DEVELOPMENT LIFE CYCLE



— SDLC PHASE 1 · GATHERING REQUIREMENTS

Gathering Requirement: Six steps



Step 3 is asking, observing and reading what already exists — not only sitting in a meeting.

Step 4 is written in the user's words, not ours.

Step 6 is where MoSCoW comes in.

MoSCoW prioritization



MUST HAVE

All the requirements that are necessary for the successful completion of the project.



SHOULD HAVE

Requirements that are important for project completion but not necessary.



COULD HAVE

Requirements that are nice to have, but have a much smaller impact when left out of the project.



WILL NOT HAVE

All the requirements that have been recognized as not a priority for the project's timeframe.

— SDLC PHASE 1 · THREE DOCUMENTS

URS, SRS and SysRS

URS

Describes the business need and the expectations of the end users.

Written as user stories or use cases, answering: who is the user, what function is needed, and why does the user want it.

Accepted by UAT.

SysRS — THE SYSTEM REQUIREMENTS

SRS — THE SOFTWARE REQUIREMENTS

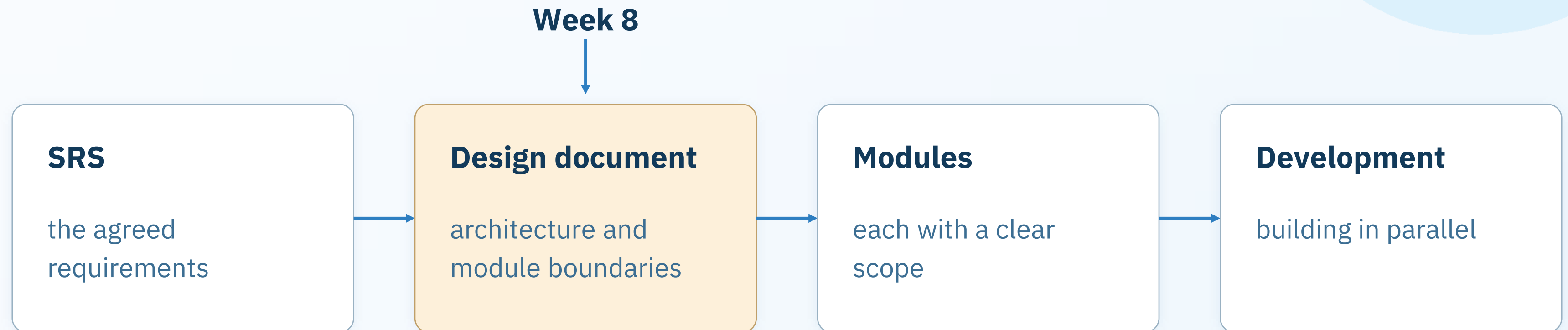
Purpose and scope · constraints, assumptions and dependencies · requirements: functional, external interface, system feature and non-functional

Capabilities · interfaces · user characteristics · policy and regulation · personnel · performance · security · hardware · acceptance criteria

The requirement that the server must sit in Thailand belongs in the SysRS, not the SRS.

— SDLC PHASES 2 AND 3

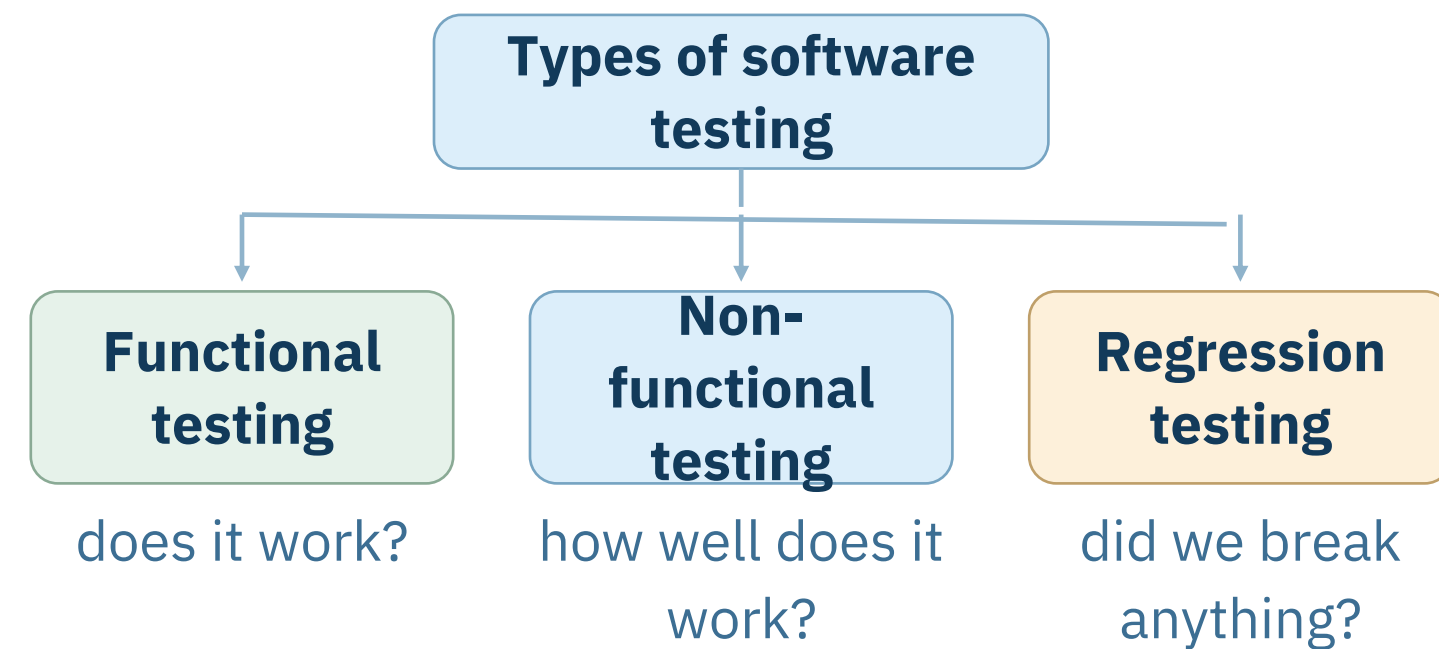
Design, then build



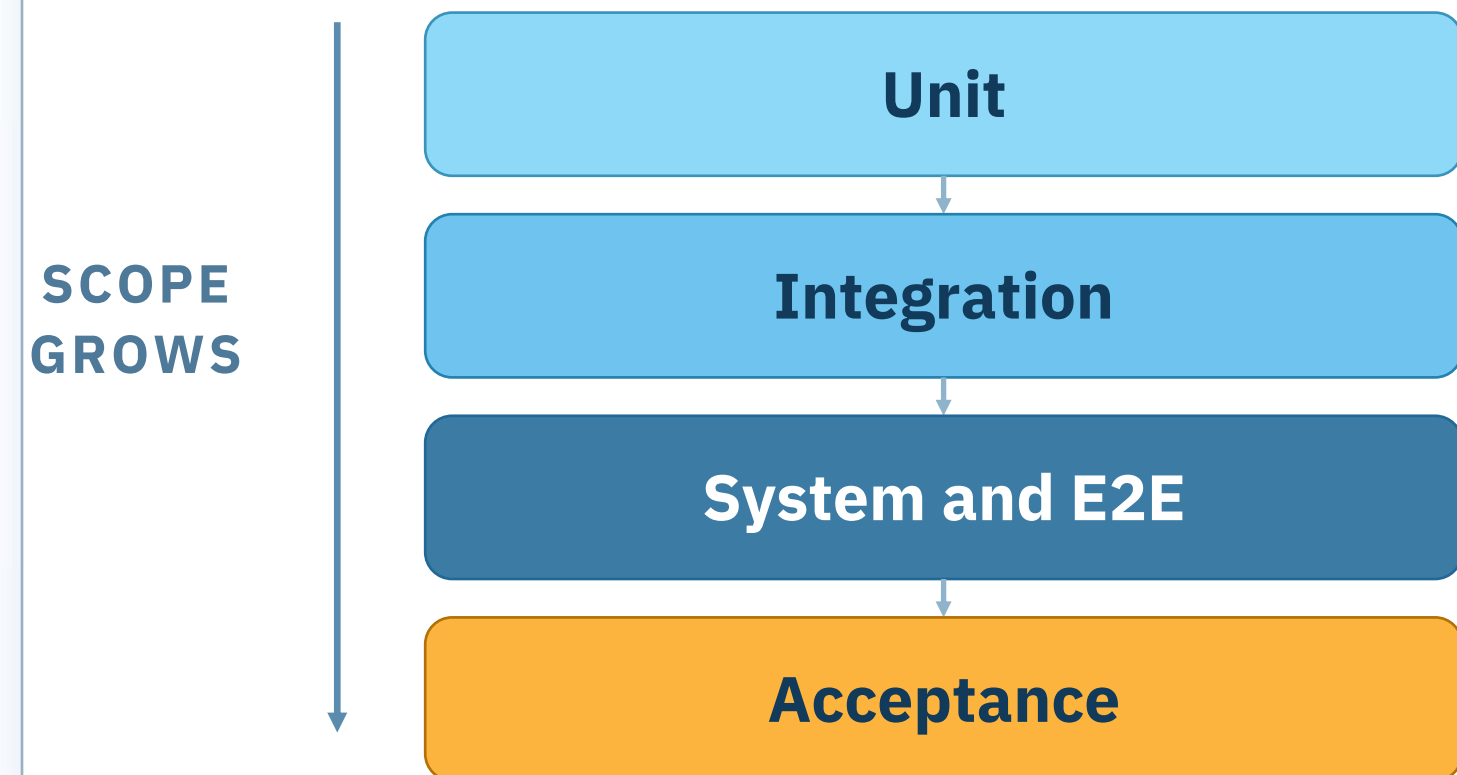
Requirements are gathered from the SRS to develop the architecture. The technical lead breaks the requirements into pieces with clear boundaries, and the design is agreed and written down before code starts.

Testing Level and Types

TYPES OF SOFTWARE TESTING



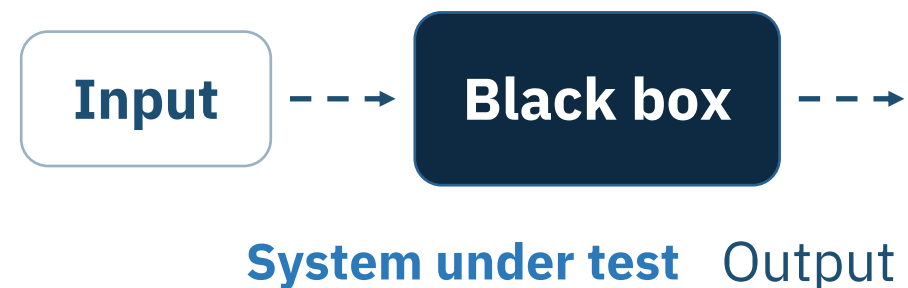
TESTING LEVELS



Each level assumes the level before it already passed.

What each type actually checks

FUNCTIONAL



Check the behaviour at the boundary, not the code inside.

NON-FUNCTIONAL

Performance

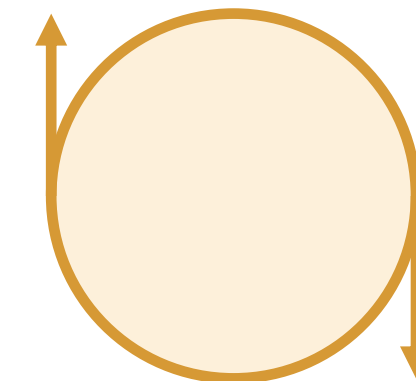
Security

Scalability

Availability

Not what the system does, but how well it does it. Each question turns into a measurable target before testing starts.

REGRESSION



Confirms changes do not break the application. Runs after fixes, or when requirements changed. Re-run the old tests so a new fix does not undo an old one.

System testing and acceptance testing

SYSTEM TESTING • and E2E

Compliance with SRS

Validate the system

Functional and non-functional

Staging environment

The whole assembled system, tested the way it will be run.

ACCEPTANCE TESTING



Users



Customers



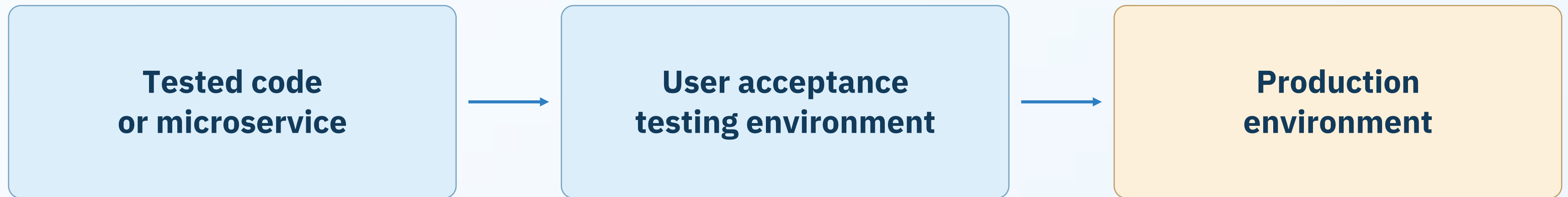
Stakeholders

Sign-off comes from the people who asked for the system.

Acceptance testing is sometimes called beta testing: real users do the testing.

— SDLC PHASE 5 · DEPLOYMENT

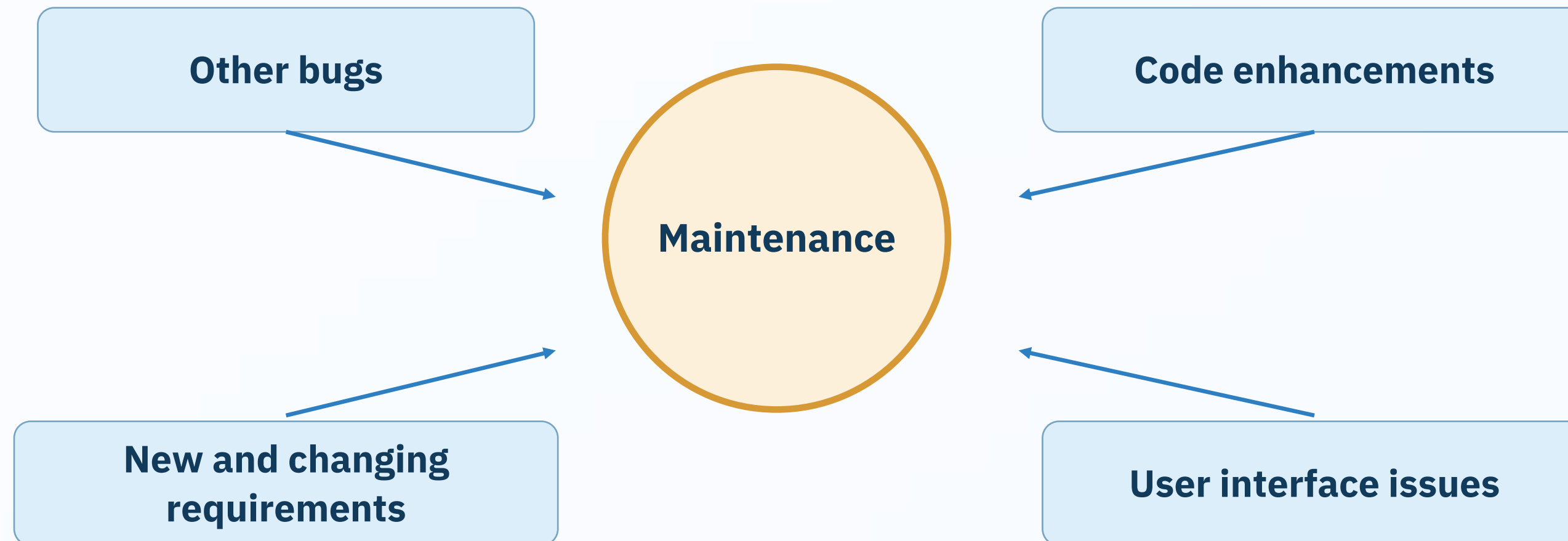
One environment at a time



Week 9 is where the automated gates and the pipeline live.

— SDLC PHASE 6 · DOCUMENTATION AND MAINTENANCE

Work keeps arriving from four directions



Three ways the same knowledge gets recorded: written assets · video assets · graphical assets

Five documents for one product

Requirements

URS, SRS and SysRS — what the product must do

Design

architecture and module boundaries

Technical

comments in the code and working papers that explain how the code works during implementation

QA

test plans, test data, test scenarios, test cases, test strategies and traceability matrices

User

FAQs, installation and help guides, tutorials and user manuals

Standard operating procedures (SOP) accompany the process documentation: step-by-step instructions for common but complex tasks, as flowcharts, hierarchies or numbered steps.

 PART 02

Tracking the plan

Where the plan meets what actually happened

— TRACKING

Following the progress

TRACKING

A method of following the progress of project activities.

DEVIATION

Anything that alters your original course of action. Deviations from the project plan can be positive or negative.

COMMONLY TRACKED ITEMS

- ◆ Project schedule
- ◆ Status of action items, key tasks and activities
- ◆ Progress toward milestones
- ◆ Costs
- ◆ Key decisions, changes, dependencies and risks to the project

— THE GANTT CHART

From last week, suitable for Waterfall

No.	Task	Owner	Start	Due	Week 1					Week 2					Week 3				
					M	T	W	T	F	M	T	W	T	F	M	T	W	T	F
1	Phase 1 — Initiation																		
1.1	Draft the project charter	PM	1 Sep	2 Sep															
1.2	Charter review and sign-off	Sponsor	3 Sep	3 Sep															
1.3	Stakeholder map	BA	2 Sep	4 Sep															
1.4	First risk log	PM	4 Sep	5 Sep															
2	Phase 2 — Planning																		
2.1	Scope and goals	BA	8 Sep	9 Sep															
2.2	Work breakdown structure	PM	9 Sep	10 Sep															
2.3	Schedule and estimates	Dev lead	10 Sep	12 Sep															
2.4	Budget	PM	11 Sep	12 Sep															
3	Phase 3 — Execution																		
3.1	Build increment 1	Dev team	15 Sep	18 Sep															
3.2	Test increment 1	QA lead	17 Sep	19 Sep															
3.3	Weekly status report	PM	12 Sep	12 Sep															
3.4	Release to the pilot group	Dev lead	19 Sep	19 Sep															

— THE ROADMAP

High level overview for the stakeholders

A roadmap states the goals and the milestones. It does not state who does what on which day.

Goal 1 A TokTickIT service desk running end to end, demonstrated in week 14

Goal 2 Every increment traceable to its specification and its tests

WEEKS 1-5 Foundations

- ◆ Vertical slice running
- ◆ Requester ticketing MVP
- ◆ Zen Green UI agreed

WEEKS 6-7 Access and roles

- ◆ Authentication
- ◆ Role-based authorization
- ◆ IT Staff queue

WEEKS 8-9 Workflow

- ◆ Actions Taken
- ◆ Status transitions
- ◆ Dashboards

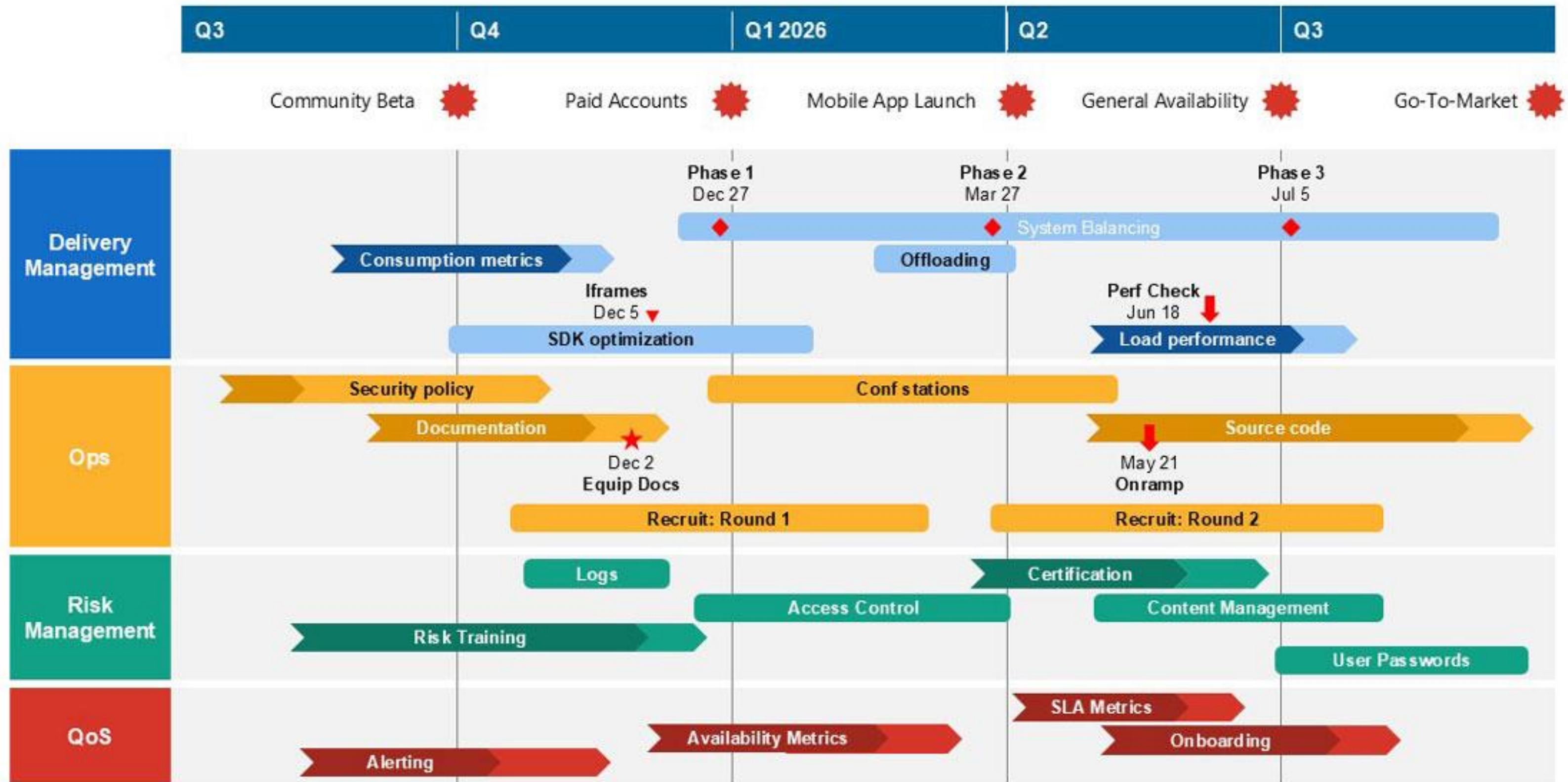
Delivery

- ◆ Team repository and CI/CD
- ◆ Cloud deployment

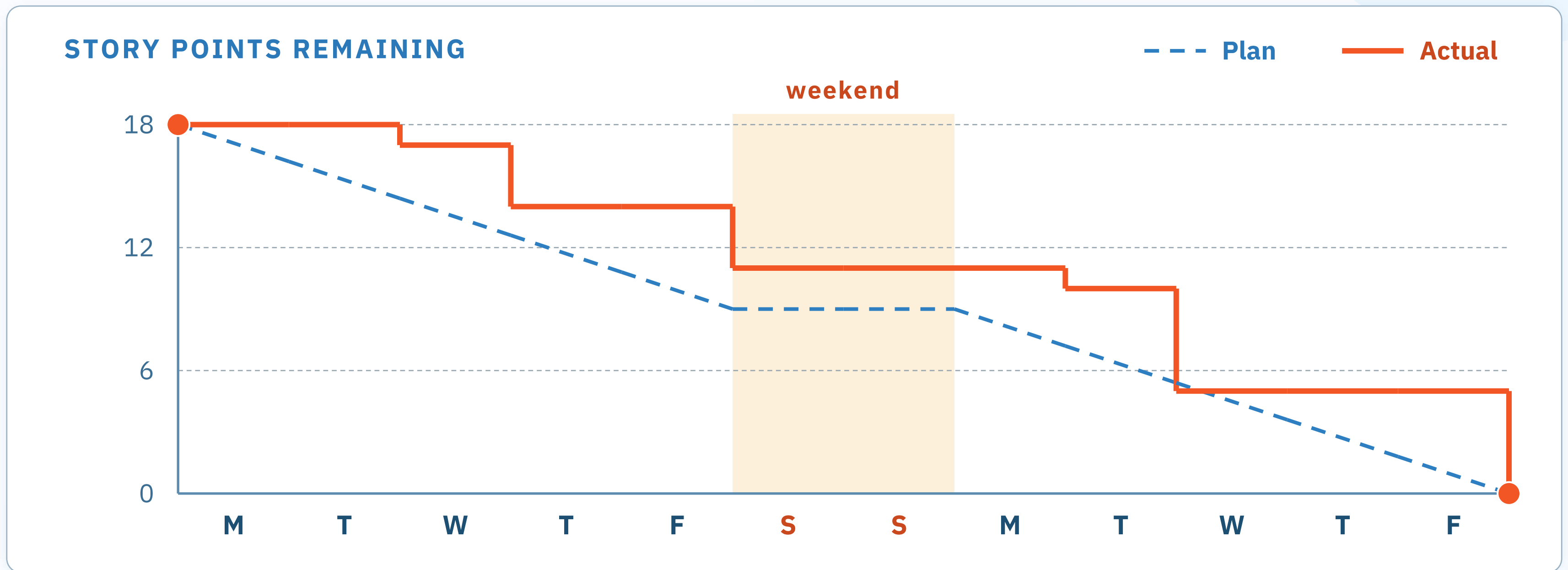
Evidence

- ◆ QA and UAT
- ◆ Metrics and report
- ◆ Final demonstration

Week 7 is the shaded column: this is the sprint you are in.



Burn down chart



 PART 03

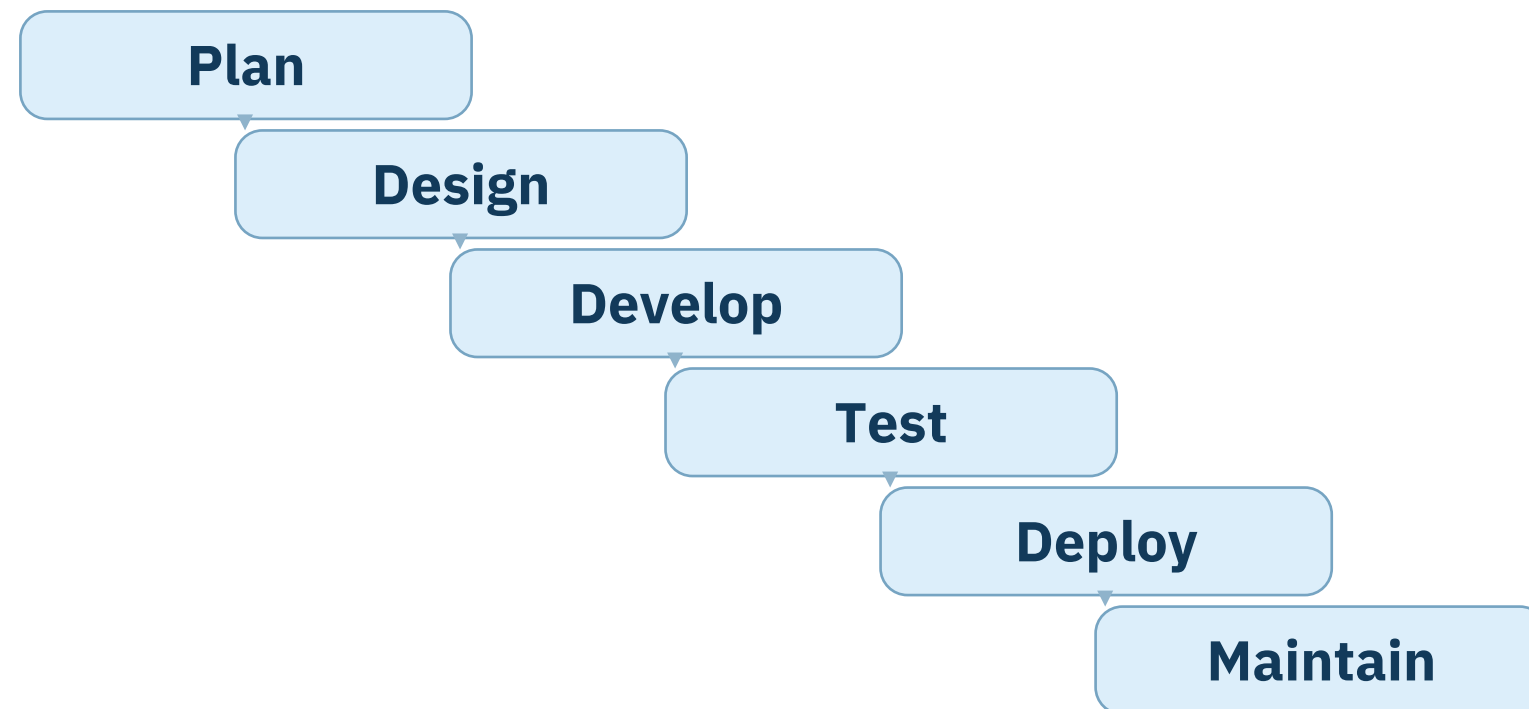
Common methods to run project

Waterfall, Agile, and the board on the wall

— TWO COMMON MODELS

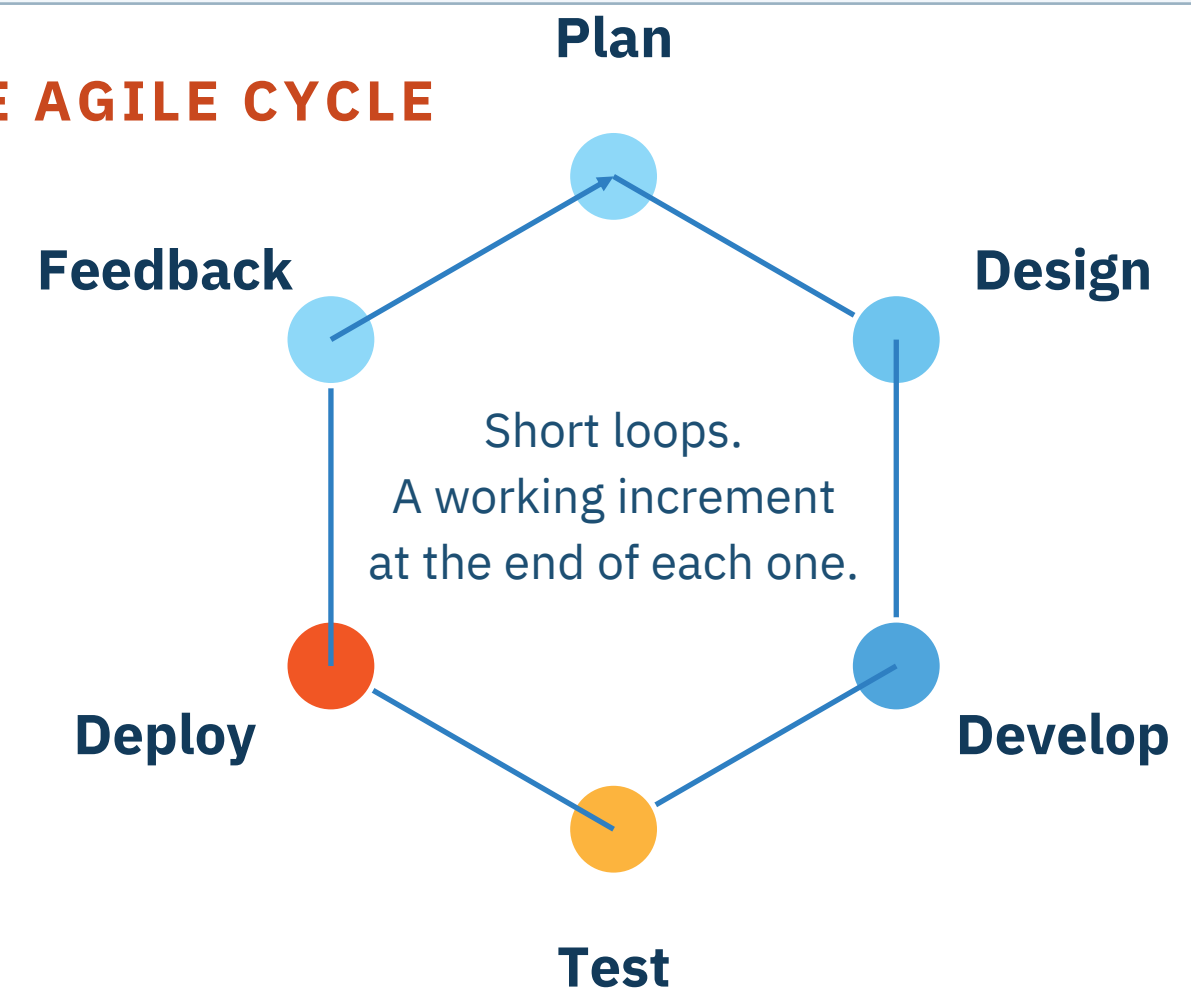
Waterfall and Agile

THE WATERFALL METHOD



One phase finishes and is signed off before the next begins.

THE AGILE CYCLE

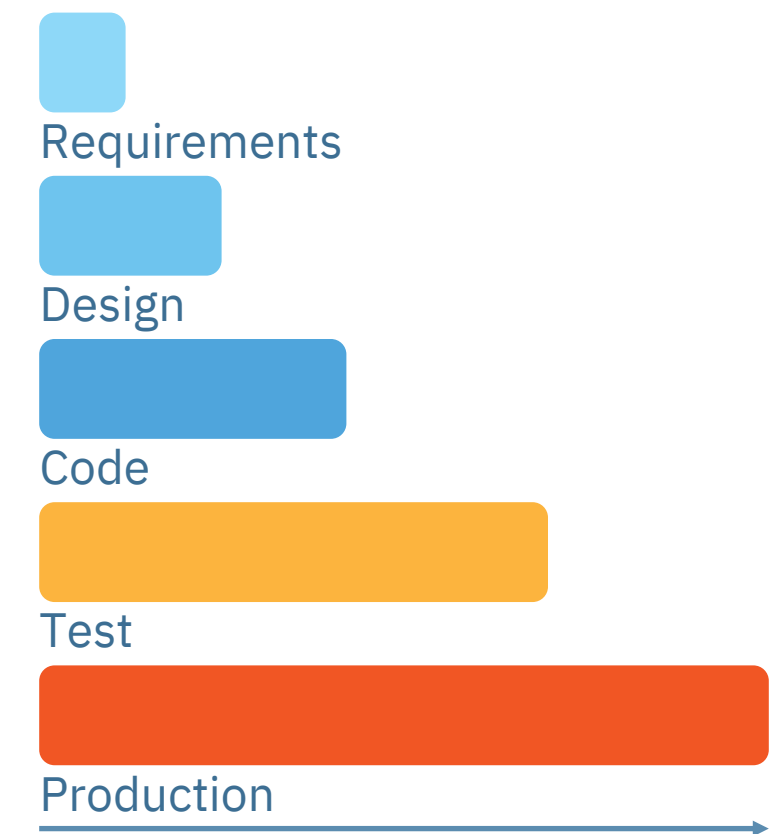


— WATERFALL

Problems with the waterfall approach

- 1 No provisions for changing requirements
- 2 No idea if it works until the end
- 3 Each step ends when the next begins
- 4 Mistakes found in the later stages are more expensive to fix
- 5 There is usually a long time between software releases

COST OF A FIX





Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

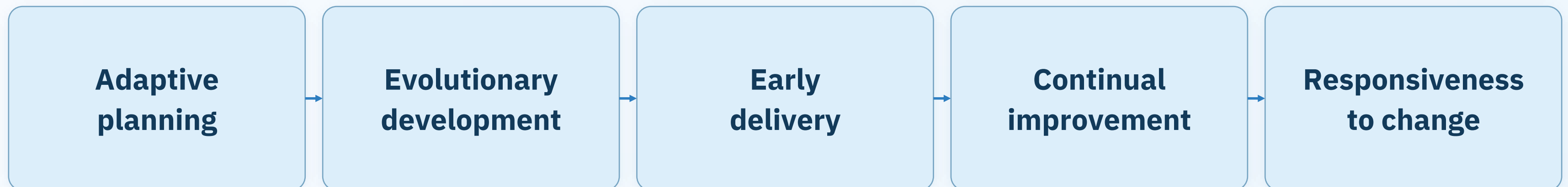
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

— WHAT IS AGILE

An iterative approach, and five habits

Agile is an iterative, collaborative approach to project management and software development, using **small cross-functional teams**, that helps teams be responsive and deliver value to their customers faster.

AGILE EMPHASISES



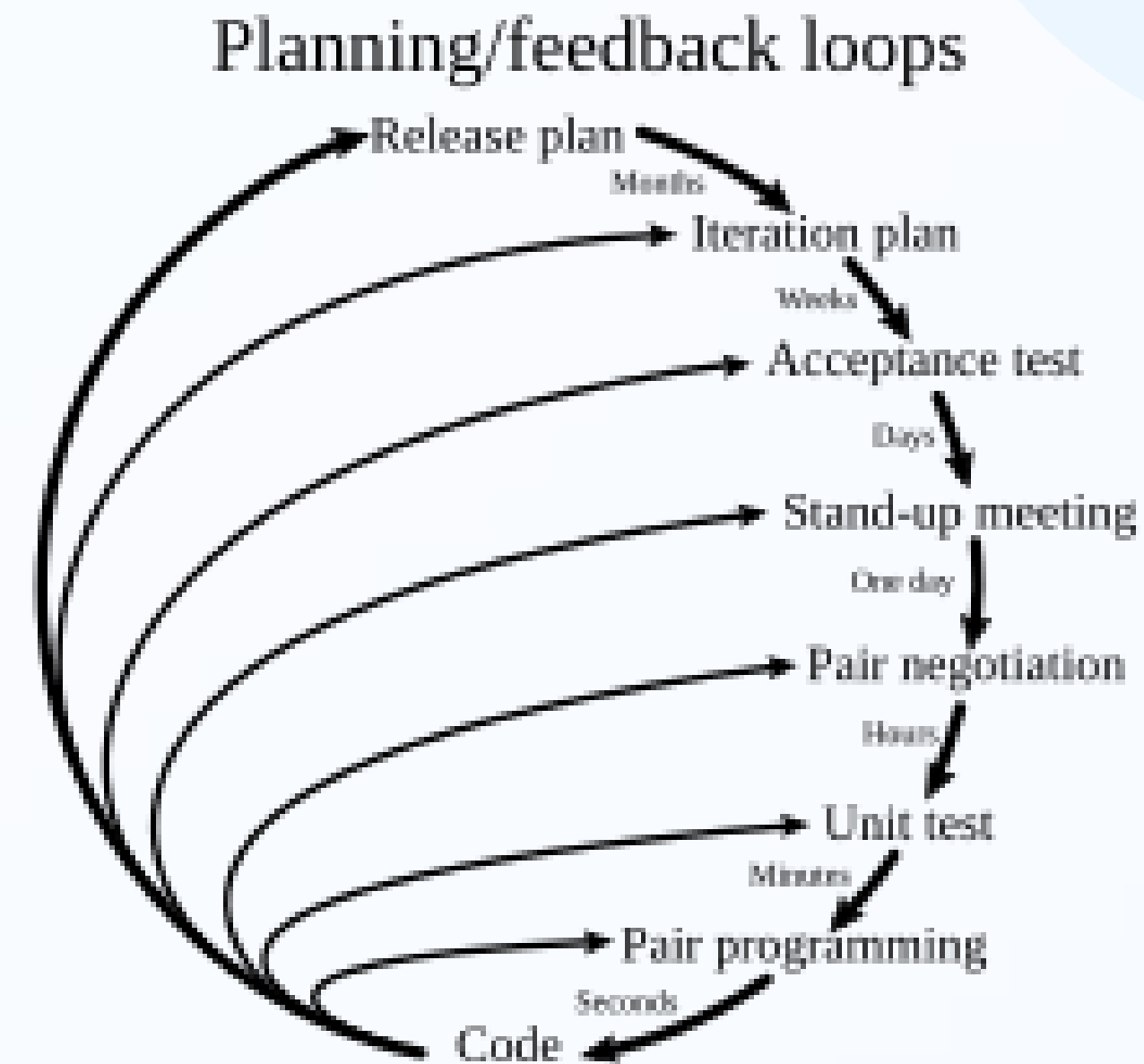
Cross-functional means the team holds every skill the increment needs — not one specialist per team.

— EXTREME PROGRAMMING

Feedback loops, from seconds to months

XP

- ◆ In 1996 **Kent Beck** introduced Extreme Programming
- ◆ Based on an interactive approach to software development
- ◆ Intended to improve software quality and responsiveness to changing customer requirements
- ◆ One of the first Agile methods



https://en.wikipedia.org/wiki/File:Extreme_Programming.svg

Five practices that make it work

- 1 Working in small batches
- 2 Minimum Viable Product
- 3 Behavior Driven Development
- 4 Test Driven Development
- 5 Pair programming

— AGILE IS NOT

Three things people get wrong



Waterfall in sprints

Agile is not a new version of a waterfall software development life cycle, where you do legacy development in sprints.



Only the developers

Agile is not just the developers working in each sprint; it involves **a cross-functional team**.



Agile project management

The Agile Manifesto does not include the term “Agile project management” — and so there are no **“project managers”** in Agile.

— KANBAN

An instruction card, sent along the line

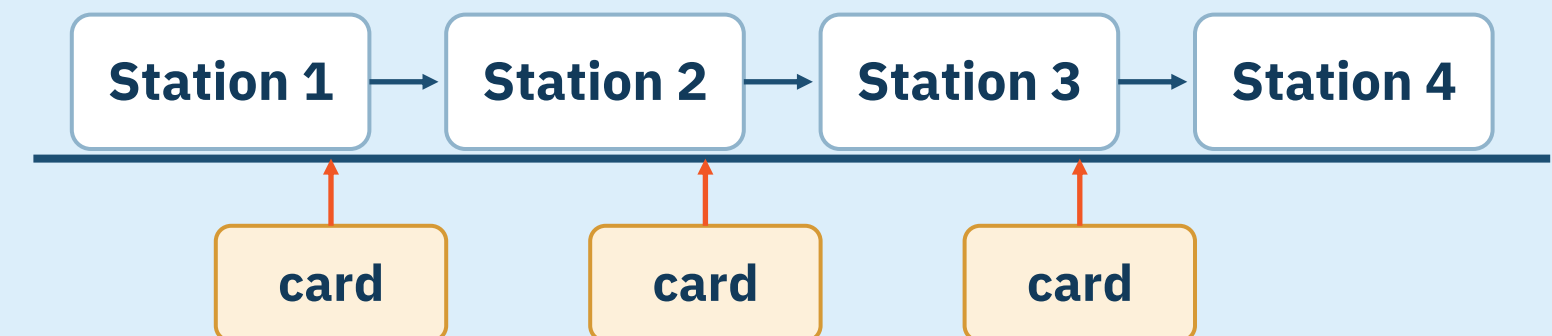
Kanban | 'kanbän | noun

Also Kanban system: a Japanese manufacturing system in which the supply of components is regulated through the use of an instruction card sent along the production line. Also, an instruction card used in a Kanban system.

ORIGIN

1970s: Japanese, literally 'billboard, sign'.

THE PRODUCTION LINE



— KANBAN

Five core principles

- 1 Visualize the workflow
- 2 Limit work in progress
- 3 Manage and enhance the flow
- 4 Make process policies explicit
- 5 Continuously improve

— KANBAN

The same board, on a wall



 PART 04

Principle of Agile

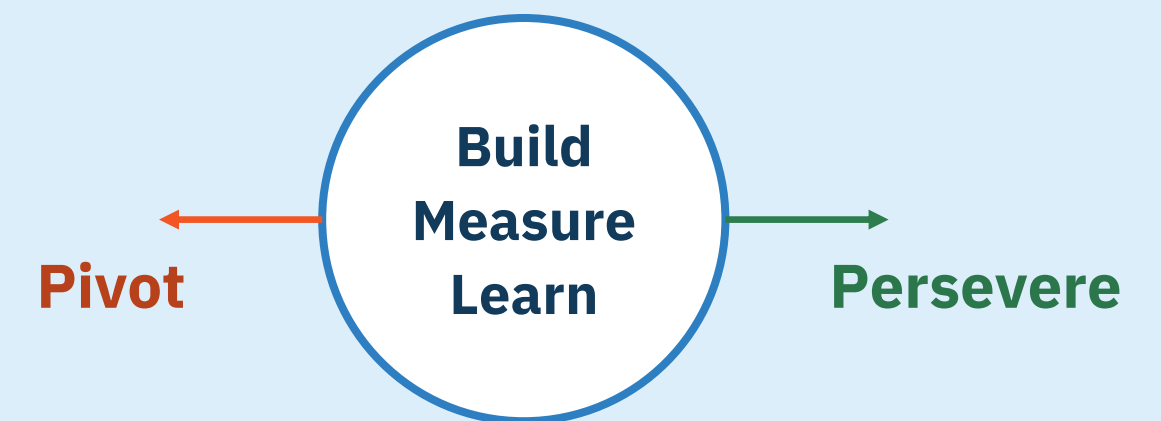
MVP, BDD, TDD and pair programming

— MINIMUM VIABLE PRODUCT

The cheapest thing that teaches you something

- ◆ MVP is the cheapest, easiest thing you can build to start testing your value hypothesis and learning
- ◆ At the end of each MVP, you decide whether to pivot or persevere

AT THE END OF EACH MVP



Pivot: the hypothesis was wrong, change direction.

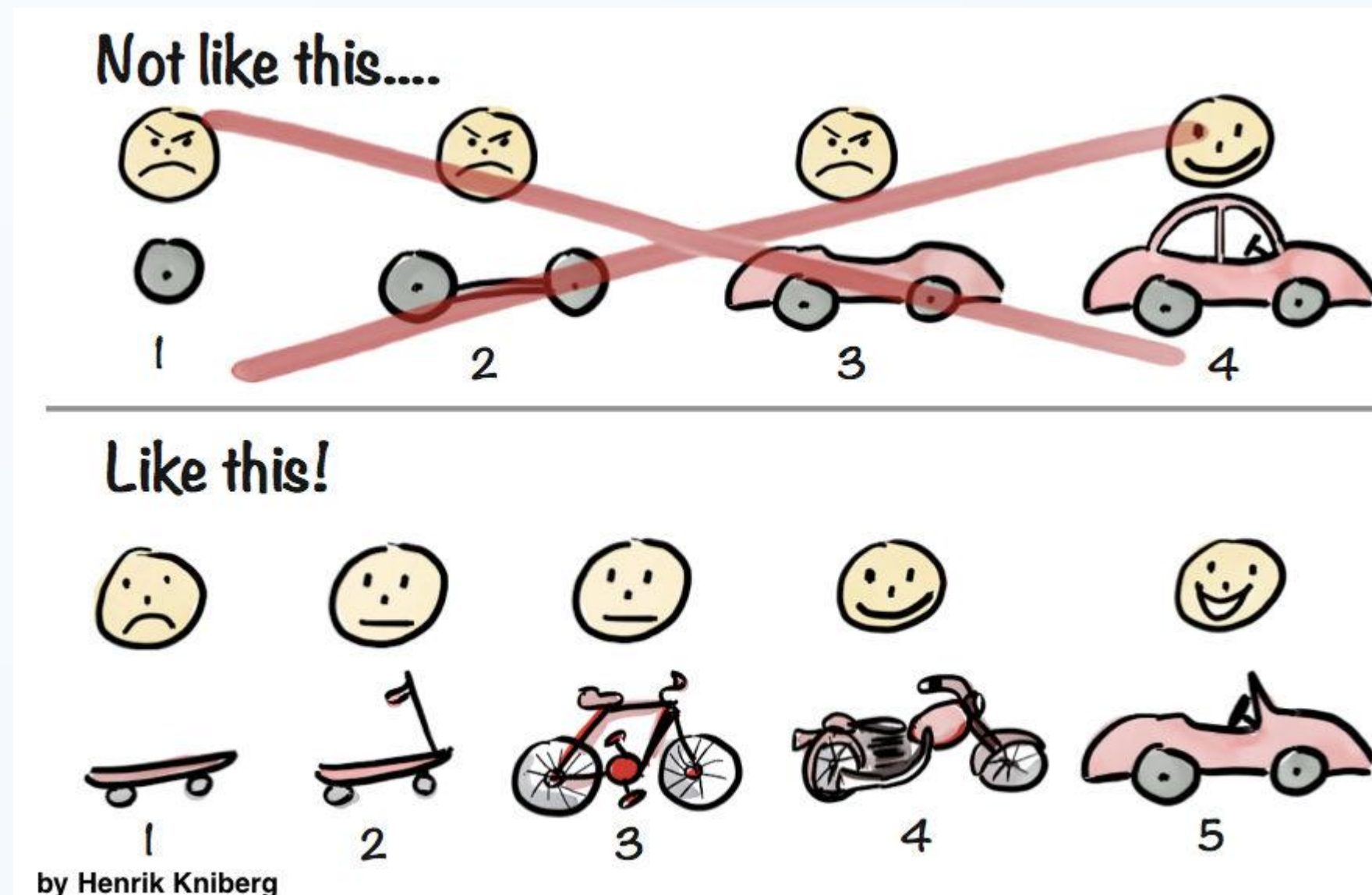
Persevere: it held, build the next increment.

MVP is not the result of “Phase 1” of a project

— MINIMUM VIABLE PRODUCT

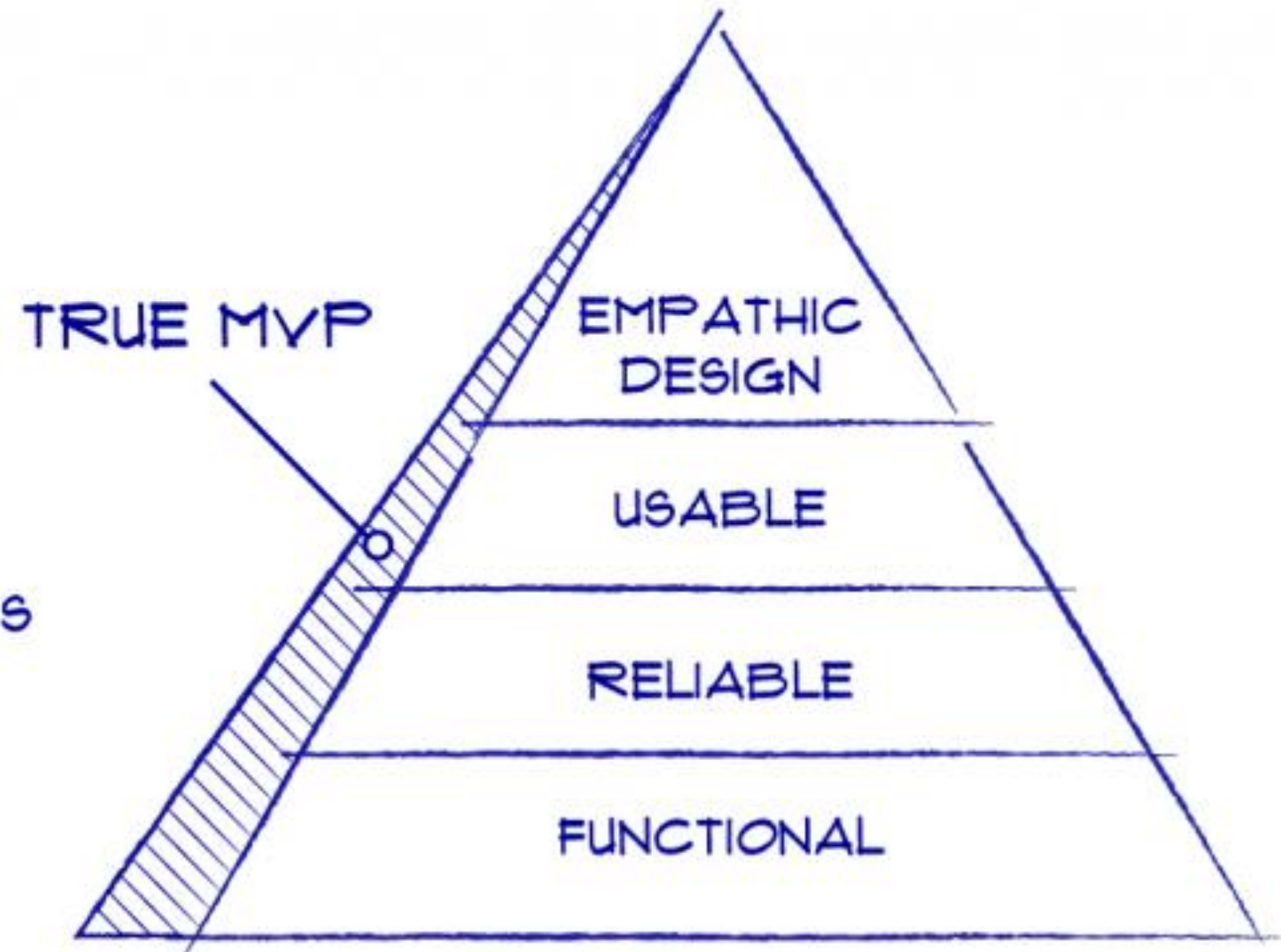
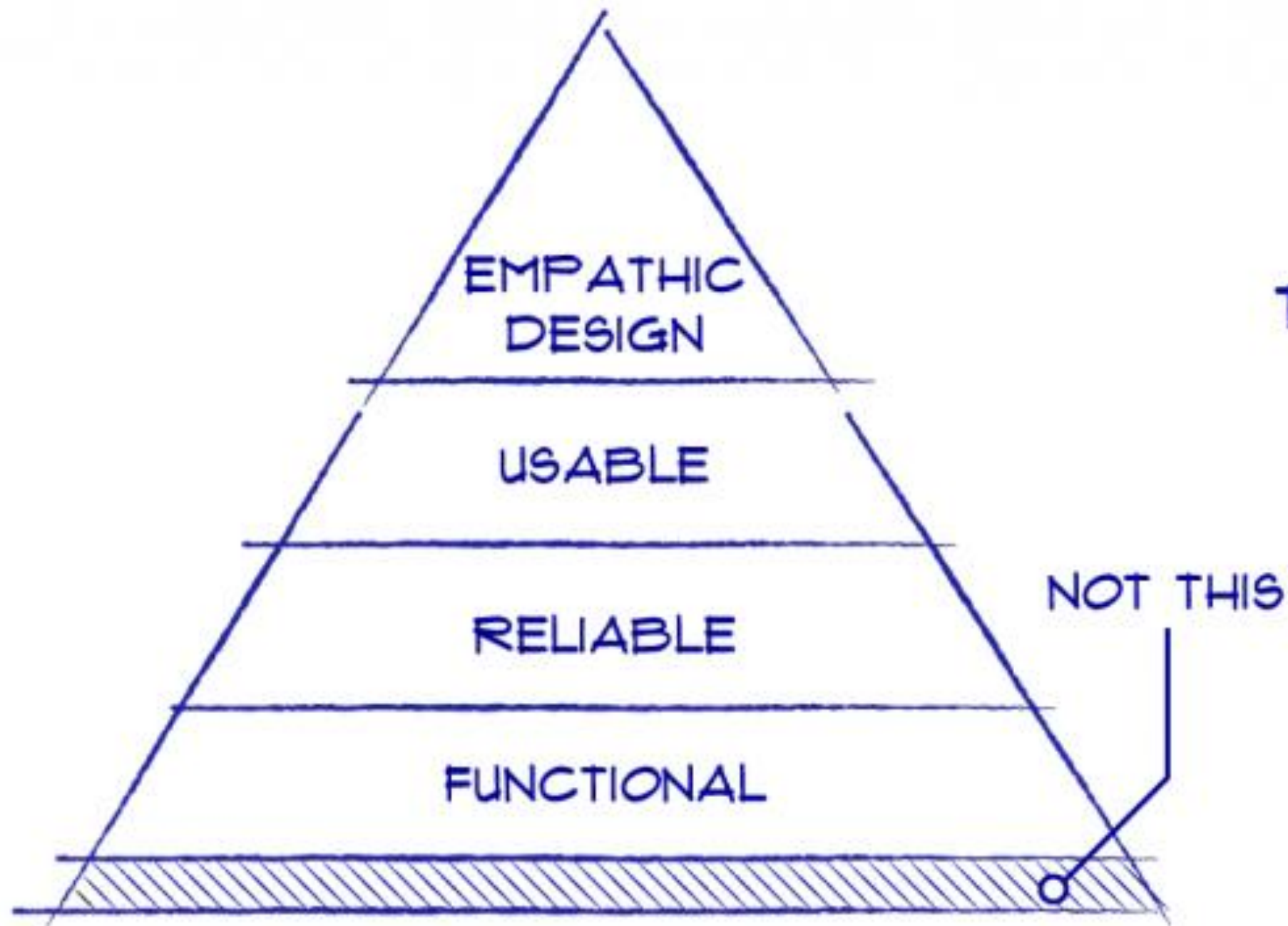
Every increment is usable on its own

The wrong way to slice: a database with no screens, then screens with no data. Neither one can be used.



<https://ixdf.org/literature/article/minimum-viable-product-mvp-and-design-balancing-risk-to-gain-reward>

MINIMUM VIABLE PRODUCT (MVP)



— BEHAVIOR DRIVEN DEVELOPMENT

The behaviour, described from the outside in

- ◆ Describes the behavior of the system from the outside in
- ◆ Used for integration testing
- ◆ Uses a single syntax to describe user stories and feature scenarios that both stakeholders and developers can understand

THE GHERKIN SYNTAX

As a <role>

I need <some functionality>

So that <some business value>

Scenario:

Given a set of preconditions

When an event occurs

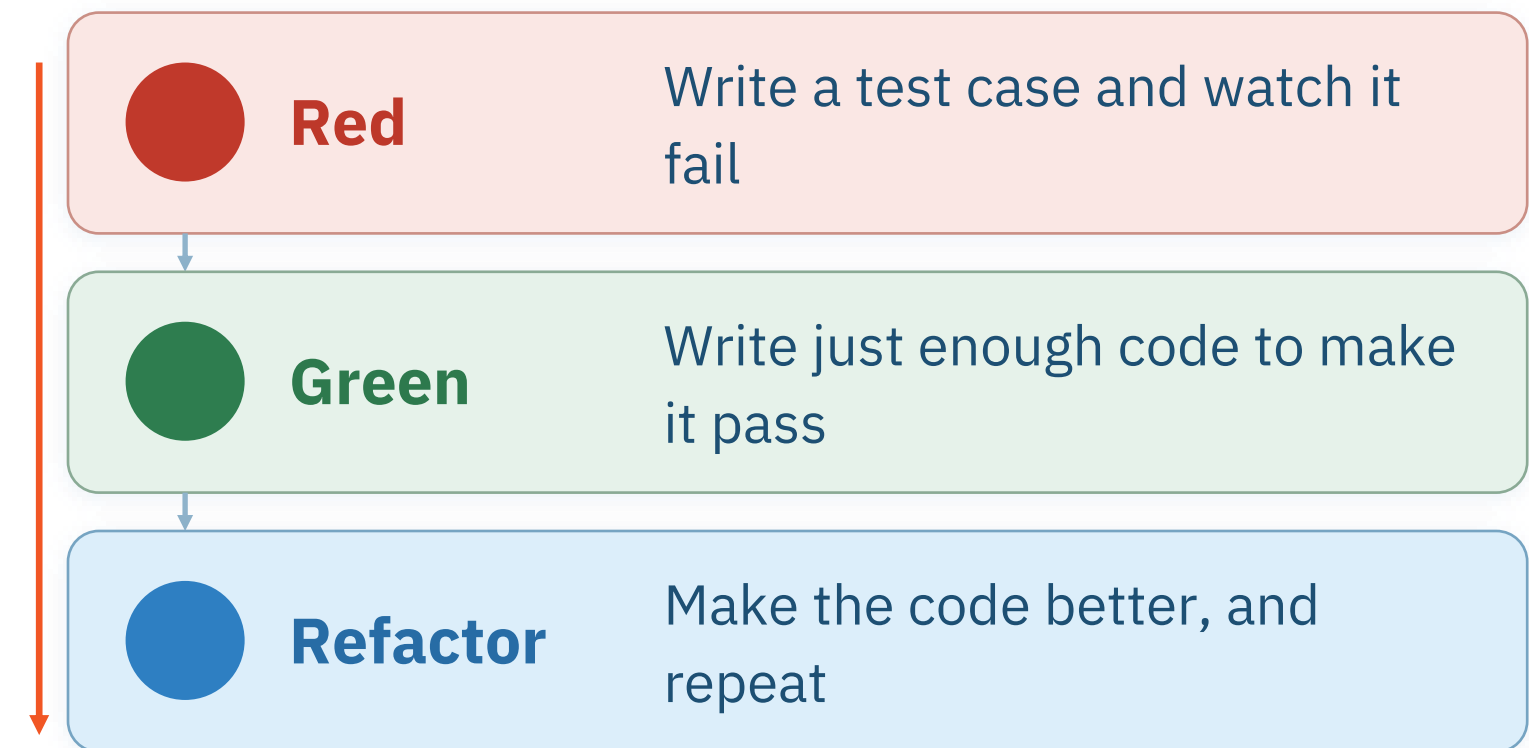
Then some outcome is observed

— TEST DRIVEN DEVELOPMENT

The tests, written from the inside out

- ◆ Tests the functions of the system from the inside out
- ◆ Test cases drive the design
- ◆ You write the tests first for the code you wish you had, then you write the code to make the tests pass

RED · GREEN · REFACTOR



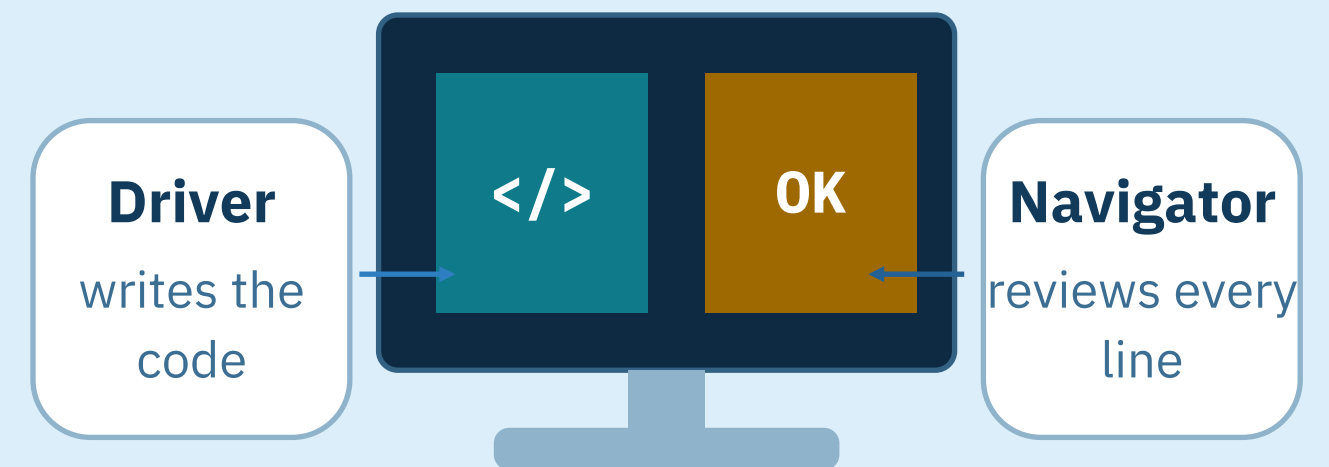
BDD and TDD are the same discipline from opposite ends: the behaviour outside, the functions inside.

— PAIR PROGRAMMING

Two programmers, one workstation

- ◆ Two programmers work together at one workstation
- ◆ One writes the code
- ◆ The other reviews each line of code as it is typed in
- ◆ The two programmers switch roles frequently (e.g., 20 min)
- ◆ Code quality increases as defects are found earlier

THE TWO SEATS



SWITCH EVERY 20 MINUTES

A review that happens while the code is written costs nothing later.

— PART 05

Scrum

One framework, three roles, five events, three artifacts

— AGILE AND SCRUM

A philosophy and a methodology

AGILE

A philosophy for doing work.

Not prescriptive: it states what to value

SCRUM

A methodology for doing work.

It adds process: named roles, named meetings, named artifacts and rules.

Every Scrum team is Agile. Not every Agile team is a Scrum team — Kanban teams are Agile too, with no sprints and no fixed roles.

— SCRUM

A management framework for incremental product development

- ◆ Is a management framework for incremental product development
- ◆ Provides a structure of roles, meetings, rules and artifacts
- ◆ Uses fixed-length iterations called sprints
- ◆ Has a goal to build a potentially shippable product increment with every iteration

THE STRUCTURE SCRUM PRESCRIBES

3 roles

product owner · Scrum master · Scrum team

5 events

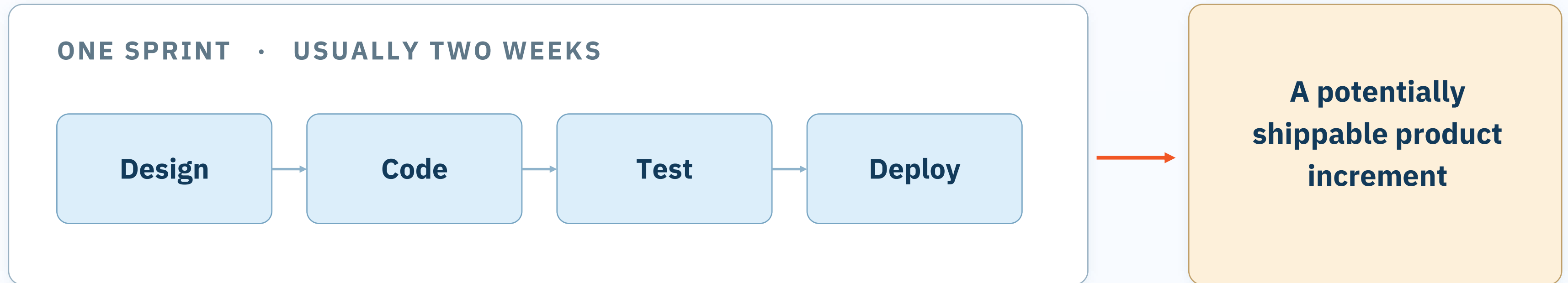
sprint planning · daily Scrum · sprint · review · sprint retrospective

3 artifacts

product backlog · sprint backlog · done increment

— THE SPRINT

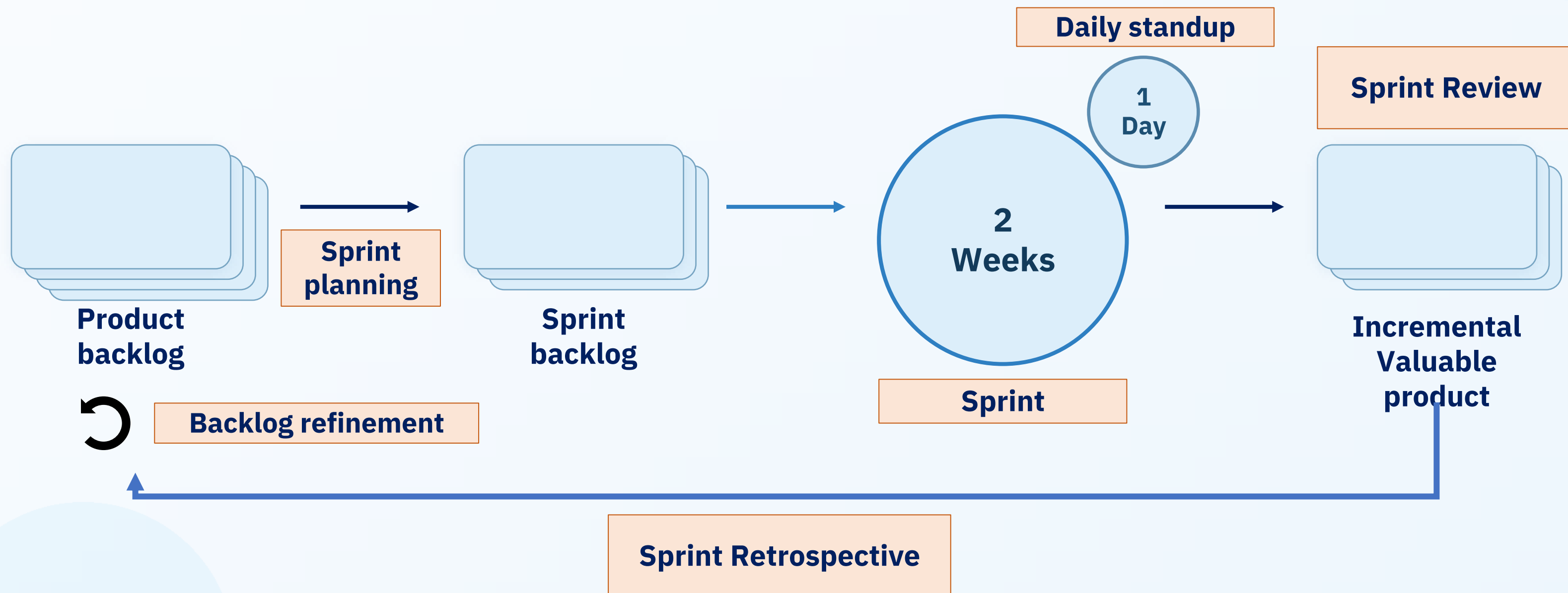
One iteration through design, code, test and deploy



Every sprint should have a goal. it is one full pass through design, code, test and deploy. If nothing can be deployed at the end of it, it was not a sprint.

— THE SCRUM PROCESS

The whole loop on one slide



— SCRUM ROLES

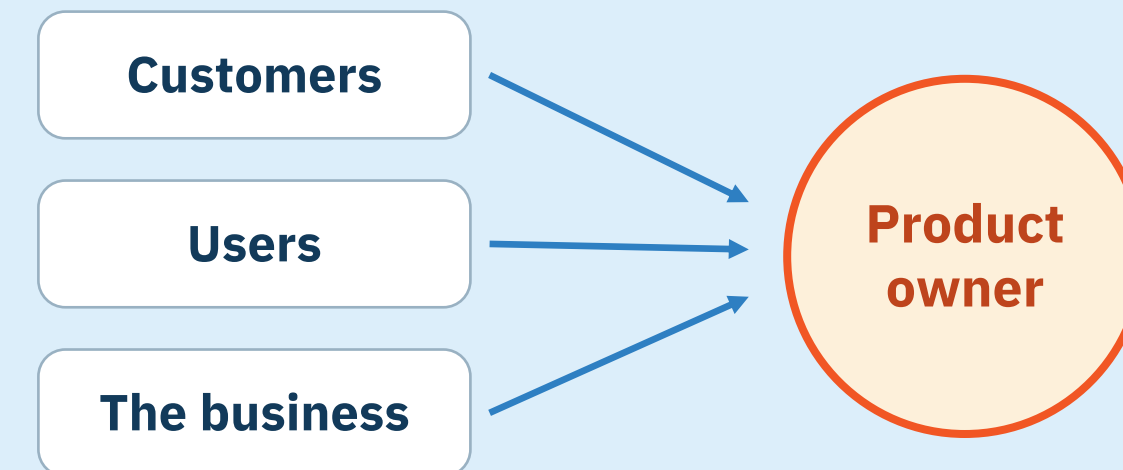
Product owner

One person who owns what gets built, and in what order

THE PRODUCT OWNER

- ◆ Represents the stakeholder interests
- ◆ Articulates the product vision
- ◆ Is the final arbiter of requirements questions
- ◆ Constantly re-prioritizes the product backlog, adjusting any expectations
- ◆ Accepts or rejects each product increment
- ◆ Decides whether to ship
- ◆ Decides whether to continue development

THE ONE VOICE FOR THE PRODUCT



Anyone may ask for work. Only the product owner decides what order it is built in.

— SCRUM ROLES

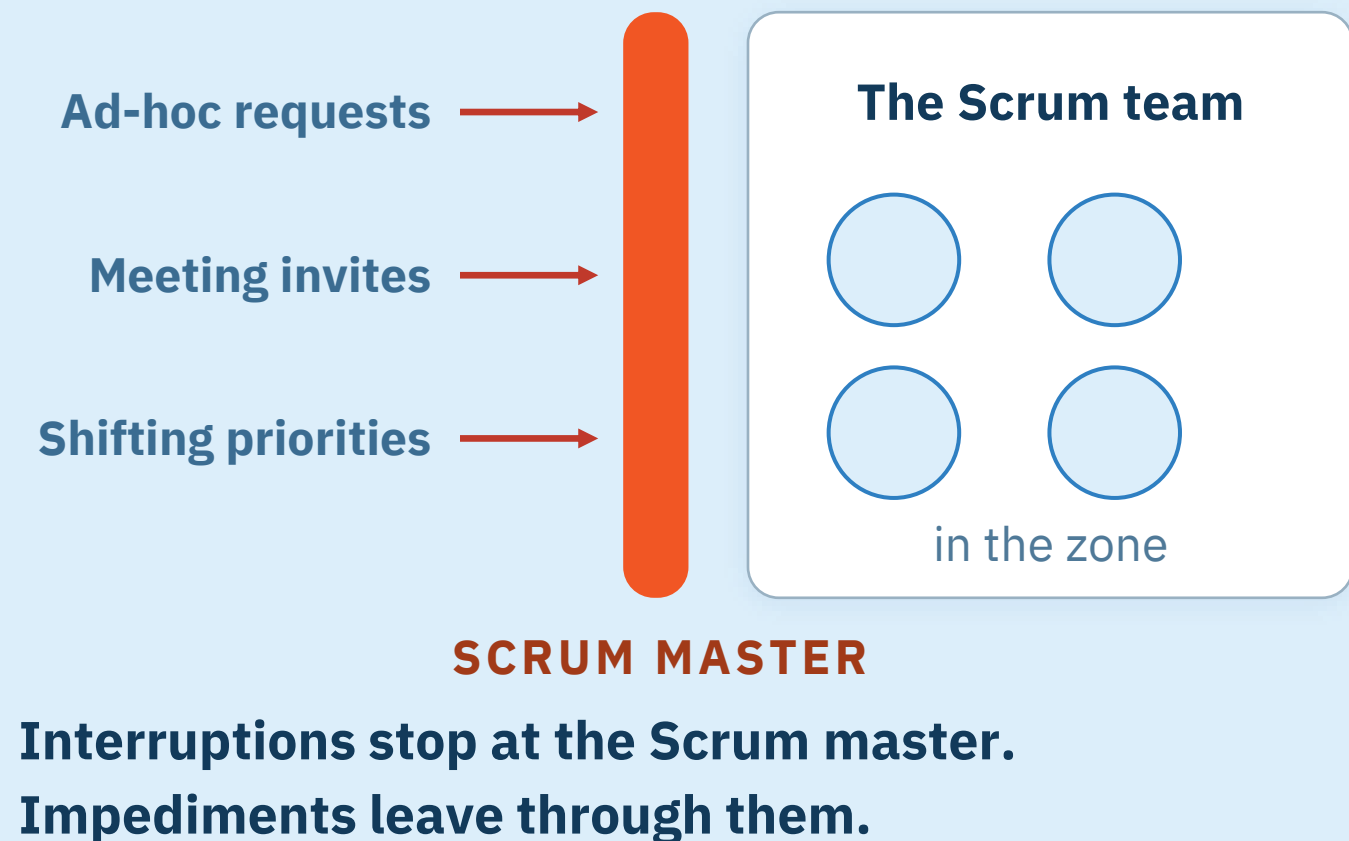
Scrum master

Serves the team and the process

THE SCRUM MASTER

- ◆ Facilitates the Scrum process
- ◆ Coaches the team
- ◆ Creates an environment to allow the team to be self-organizing
- ◆ Shields the team from external interference to keep it “in the zone”
- ◆ Helps resolve impediments
- ◆ Enforces sprint timeboxes
- ◆ Captures empirical data to adjust forecasts

WHAT “SHIELDS THE TEAM” LOOKS LIKE



Has no management authority over the team — the Scrum master cannot assign work and cannot decide what gets built.

— SCRUM ROLES

Scrum team

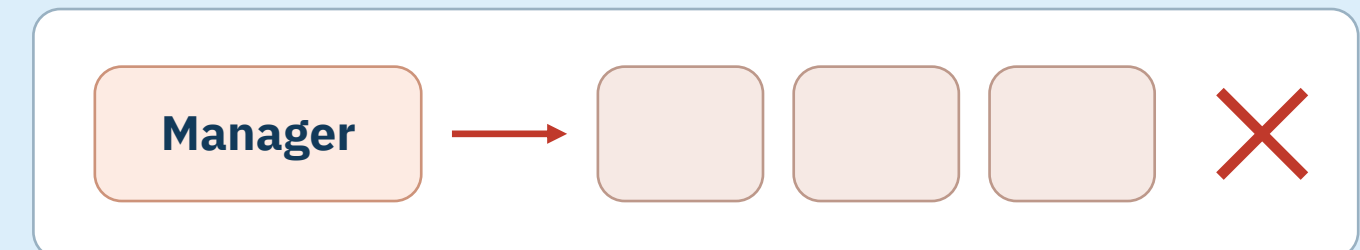
A cross-functional team of developers, testers, business analysts, domain experts and others

THE SCRUM TEAM

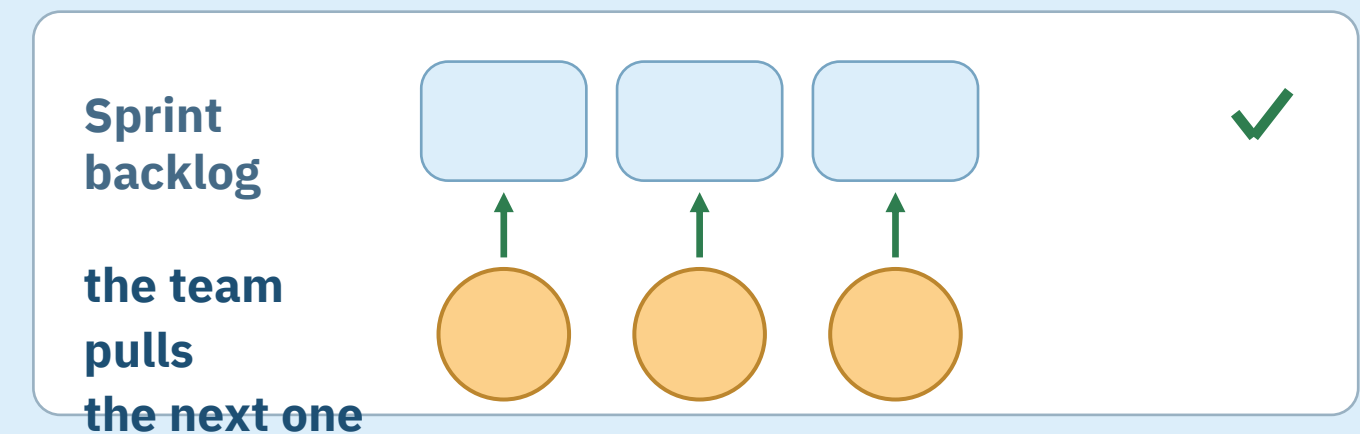
- ◆ **Self-organizing**
There are no externally assigned roles
- ◆ **Self-managing**
They self-assign their own work
- ◆ **Negotiates commitments with the product owner**
one sprint at a time
- ◆ **Has autonomy regarding how to reach commitments**

HOW WORK IS TAKEN, NOT GIVEN

NOT THIS



THIS



The product owner decides what and in what order. The team decides who does it and how.

— THE SCRUM TEAM

Who is on it, and what makes it work

A CROSS-FUNCTIONAL TEAM CONSISTING OF

Developers

Testers

Business analysts

Domain experts

Others

MEMBERSHIP

At most around 7 ± 2 collaborative members

Too much and communication become difficult

Co-located

most successful when located in one team room, particularly for the first few sprints

Dedicated

most successful with long-term, full-time membership

— ARTIFACTS AND EVENTS

Three things, five meetings

SCRUM ARTIFACTS

- 1 Product backlog**
every story not yet in a sprint
- 2 Sprint backlog**
the stories this sprint committed to
- 3 Done increment**
what exists at the end of the sprint

SCRUM EVENTS

- | | |
|-----------------------------|----------------------------|
| Backlog refinement | Before every sprint |
| Sprint planning | At the start of the sprint |
| Daily stand up | Everyday in a sprint |
| Sprint review | Demo the sprint |
| Sprint retrospective | once, after the review |

— BENEFITS OF SCRUM

What teams get out of it

1

Higher productivity

work in progress is limited, so work finishes

2

Better product quality

every increment is tested, not only the last one

3

Reduced time to market

small pieces ship sooner than one big piece

4

Increased stakeholder satisfaction

they see the product in weeks, not months

5

Better team dynamics

everything is visible, so nothing needs chasing

6

Happier employees

autonomy over how the work gets done

 PART 06

Beyond the Scrum Team

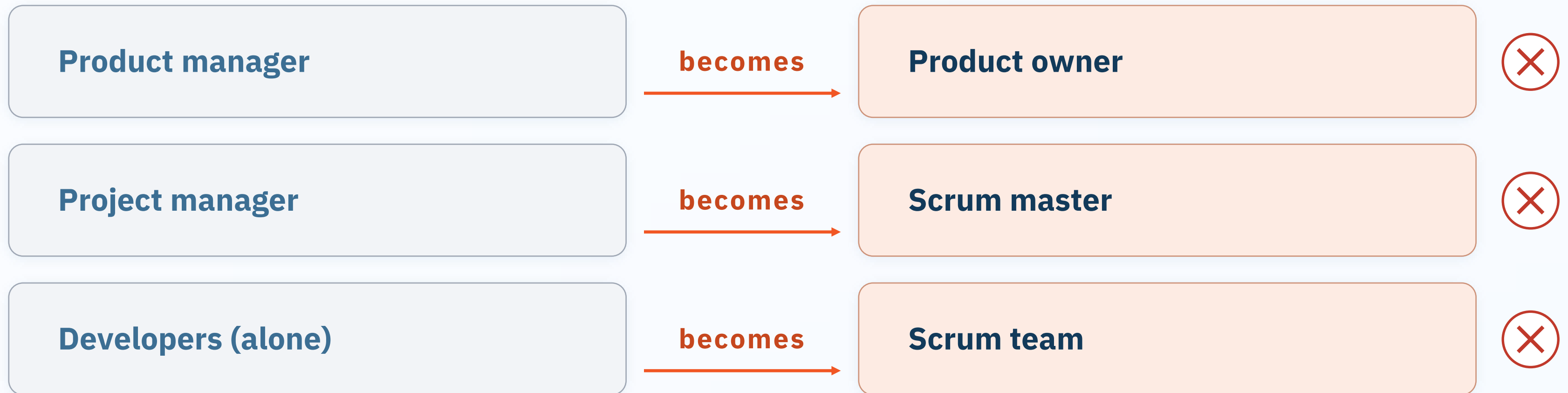
Roles, the organisation, and the ways both go wrong

Scrum vs Kanban

	Scrum	Kanban
Workflow	Fixed length sprints	Continuous flow
Release methodology	End of each sprint	Continuous delivery
Roles	Product Owner, Scrum Master, Development Team	No existing roles (optional Agile Coach)
Key metrics	Velocity	Cycle time
Change philosophy	Teams should strive to not make changes to the sprint forecast during the sprint	Change can happen at any time

— FORMULAS FOR FAILURE

Renaming people does not change how work flows





“

Until and unless business leaders accept the idea that they are no longer managing projects with fixed functions, timeframes, and costs, as they did with waterfall, they will struggle to use agile as it was designed to be used.

 Bob Kantor, Founder, Kantor Consulting Group, Inc.

Loosely coupled, tightly aligned

THE STARTING POINT

- ◆ **Proper organization is critical to success**
- ◆ **Existing teams may need to be reorganized**

Reorganising is the expensive part, and the part most organisations skip.

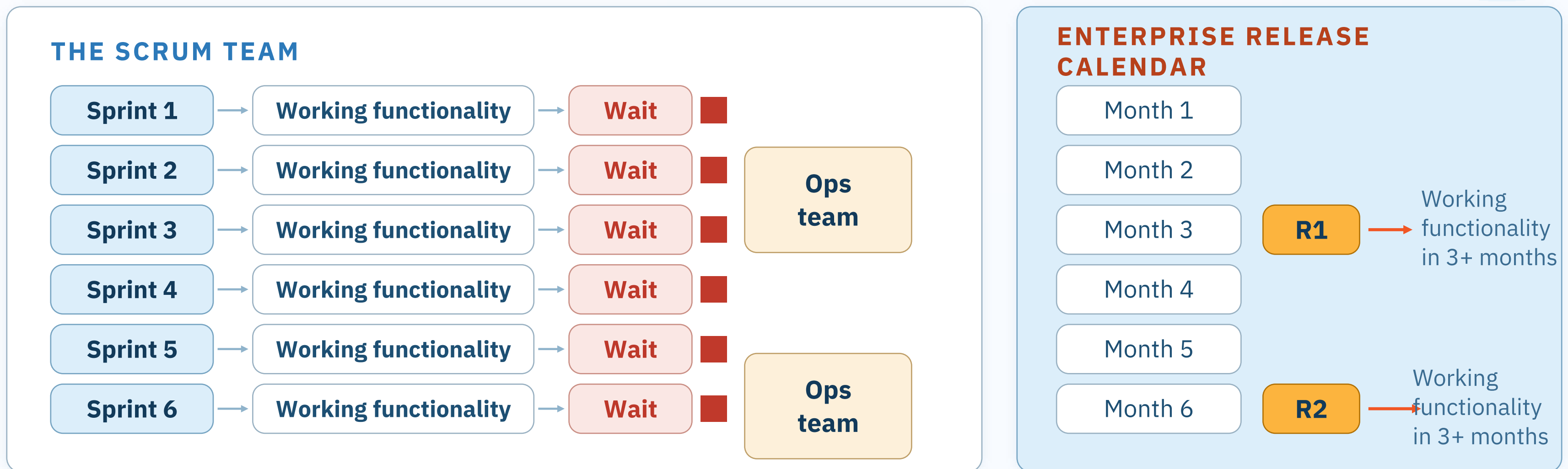
HOW TEAMS SHOULD BE ALIGNED

- 1 **Teams are loosely coupled, tightly aligned**
- 2 **Each team has its own mission aligned with the business (like a “mini startup”)**
- 3 **Teams have end-to-end responsibility for what they build**
- 4 **The long-term mission is usually around a single business domain**

Loosely coupled means one team can ship without waiting for another. Tightly aligned means they are all shipping toward the same thing.

— THE ENTIRE ORGANISATION MUST BE AGILE

A fast team behind a slow release calendar



The team finished in two weeks. The customer got it in three months. The bottleneck moved, it did not disappear.

The two sets of goals line up

The Goals of Agile	The Goals of DevOps
Develop software faster	Accelerate time to market
Be responsive to changes	Improve value by more closely aligning development, IT operations, and the business
Obtain higher quality results	Increase productivity



“

The term (development and operations) is an extension of agile development environments that aims to enhance the process of software delivery as a whole.

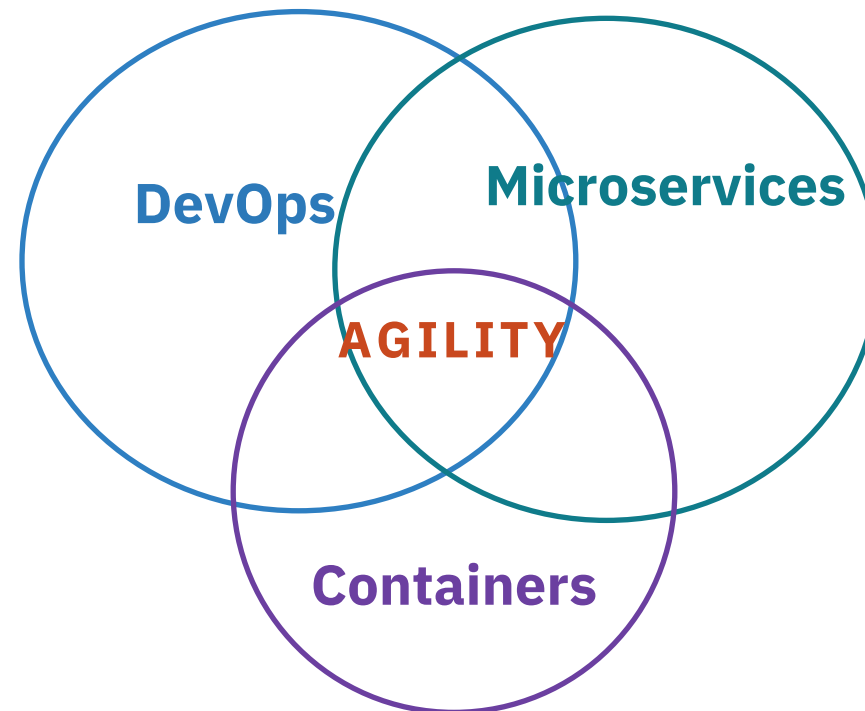
— Patrick Debois, 2009

— AGILITY

DevOps Three pillars

Cultural change ·
automated pipeline

Portability ·
developer centric ·
ecosystem enabler ·
fast startup
infrastructure as
code · immutable
infrastructure



Loose coupling ·
RESTful APIs ·
designed to resist
failures · test by
breaking, fail fast

APPLICATION EVOLUTION

Delivery

Waterfall

Agile

DevOps

Architecture

Monoliths

SOA

Microservices

Infrastructure

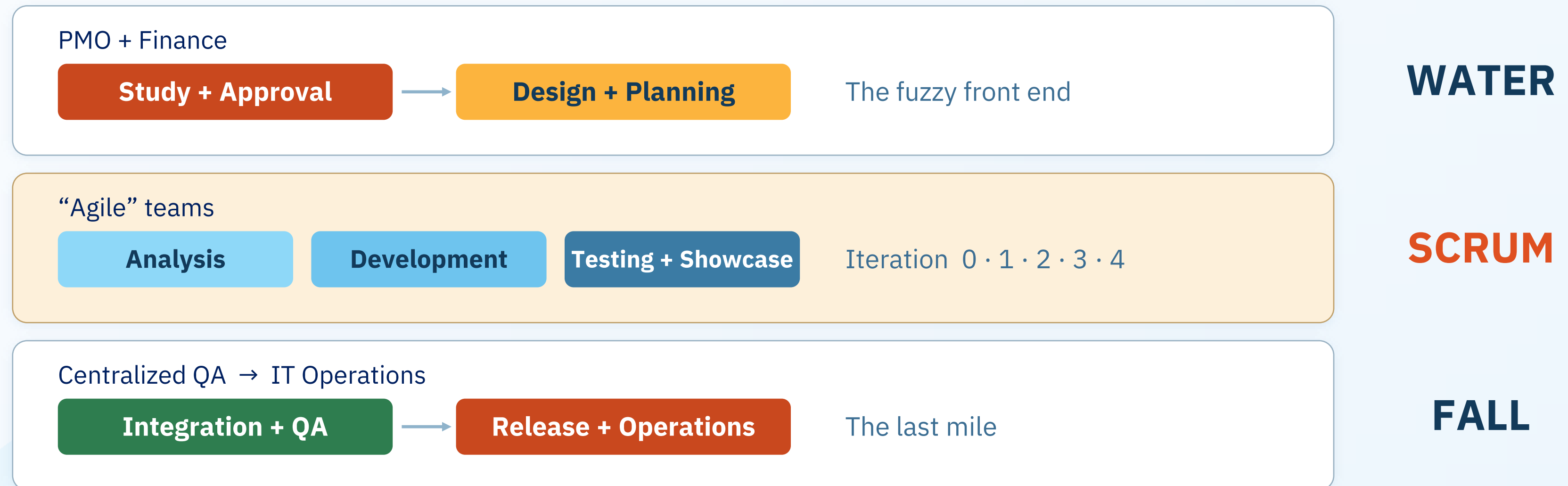
Physical
servers

Virtual
machines

Containers

— WATER-SCRUM-FALL

What organizations actually run



The team in the middle is Agile. The two ends are not, and they set the actual delivery date.

 PART 07

User stories and story points

— USER STORIES

Definition and contents

A user story represents a small piece of business value that a team can deliver in an iteration.

STORY CONTENTS · STORIES SHOULD CONTAIN

1

A brief description of the need and business value

why anyone wants it

2

Any assumptions or details

what the team already knows

3

The definition of “done”

the acceptance criteria

— STORY DESCRIPTION

The three-line template

User stories document a persona requesting a function to achieve a goal.

As a <some role>

I need <some function>

So that <some benefit>

THE PERSONA

who is asking — a role, never a person's name

THE FUNCTION

what the system must let them do

THE GOAL

why it is worth building at all

Gherkin syntax

It is critical to document the definition of “done”.

The Gherkin syntax is one line each for the precondition, the event and the outcome — and it turns straight into a test case.

Given <some precondition>

When <some event happens>

Then <some outcome>

the state the system is in before anything happens

— A SAMPLE STORY

A story as a GitHub issue

As a Marketing Manager

I need a list of customer names and emails

So that I can notify them of marketing promotions

Assumptions and Details:

- We maintain customer emails
- Customers have opted-in to promotions

Acceptance Criteria:

Given there are 100 customers in the database

And 90 have opted into email promotions

When I request the customer email list

Then I should see a list of 90 customer emails

Assignees

- No one

Labels

- None yet

Milestone

- No milestone

Estimate

- No estimate

— STORY QUALITY

Bill Wake's INVEST

I	Independent	can be built without waiting for another story
N	Negotiable	the “how” is still open for the team to decide
V	Valuable	somebody outside the team wants the result
E	Estimable	the team knows enough to give it a size
S	Small	fits inside one iteration, ideally a few days
T	Testable	has criteria that can be checked

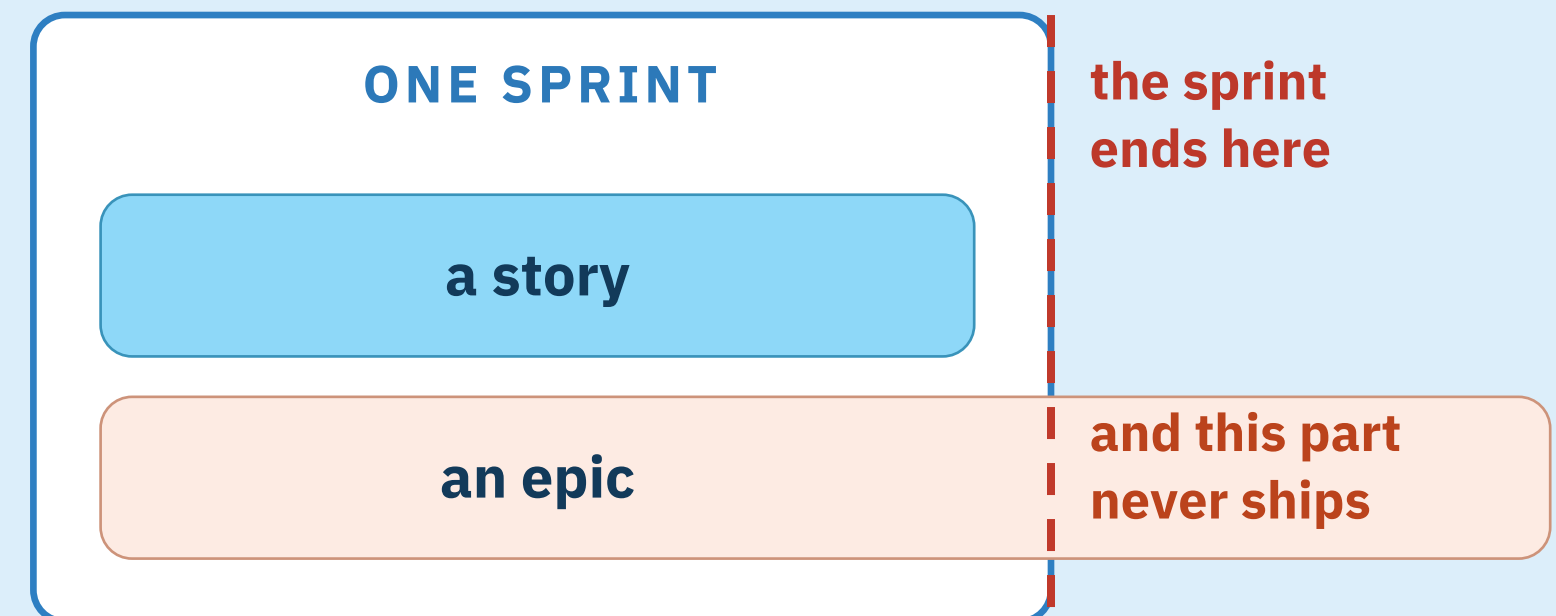
— EPICS

Definition of an epic

AN EPIC IS

- ◆ A big idea
- ◆ A user story that is bigger than a single sprint
- ◆ A user story that is too big to estimate on its own

WHY IT CANNOT GO INTO A SPRINT



An epic is not a bad story. It is a story that has not been split yet — and until it is split, no honest number can be put on it.

— EPIC SPLITTING

Epics in the backlog

- ◆ When a story is too large in scope it is considered an epic
- ◆ Backlog items tend to start as epics when they are lower priority and less defined
- ◆ For sprint planning, epics should be broken down into smaller stories

BREAKING ONE DOWN

EPIC · Dashboards for every role

Requester dashboard: open, waiting, recently resolved

IT Staff dashboard: unassigned, mine, by status

Administrator dashboard: reuse IT Staff, plus account counts

Every card links to the list it summarises

— STORY POINTS

Definition of a story point

STORY POINT

- ◆ A metric used to estimate the difficulty of implementing a given user story
- ◆ An abstract measure of overall effort

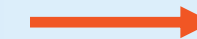
Abstract means the number has no unit. It only has meaning next to the team's other numbers.

ONE STORY, ONE NUMBER

As a User

I need a counter

So that I can count



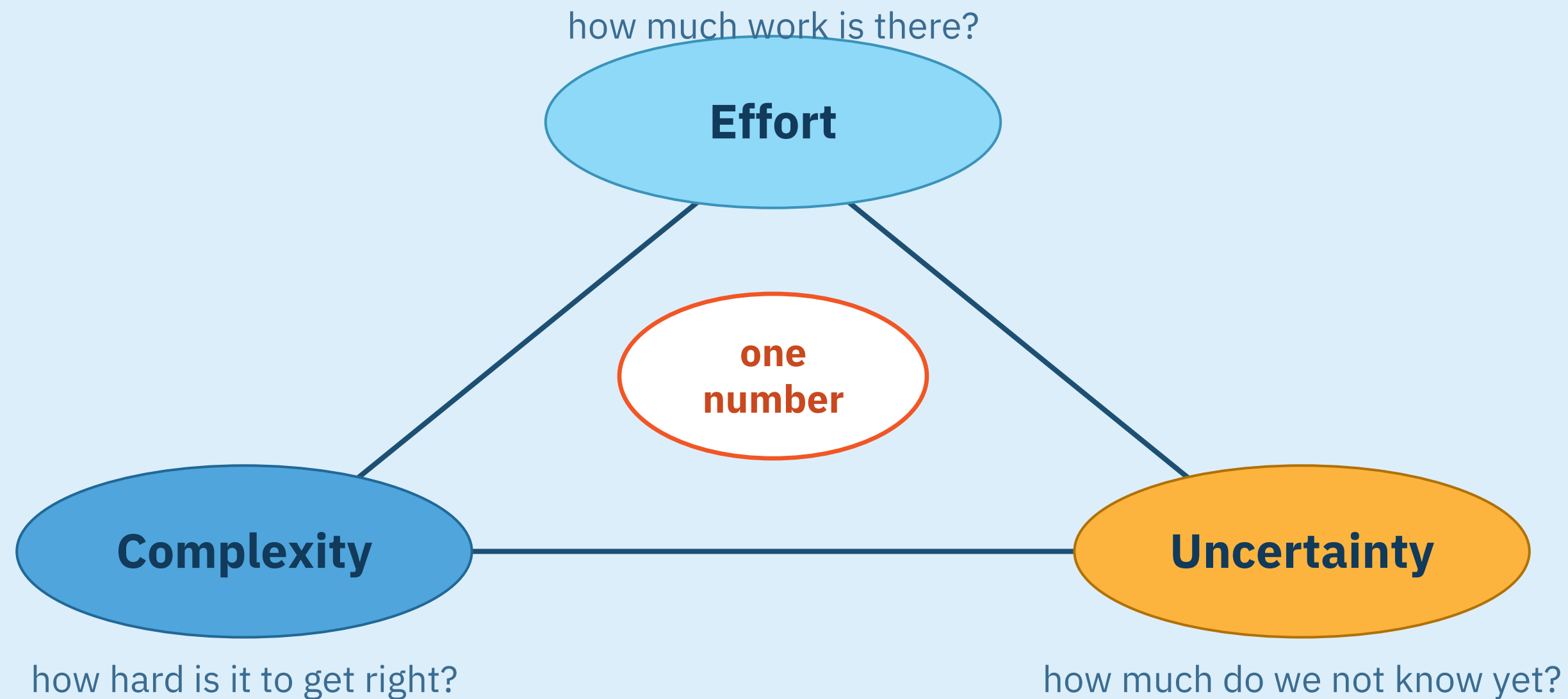
5

STORY POINTS

Not 5 hours. Not 5 days. Five — compared with the other stories this team has sized.

— STORY POINTS

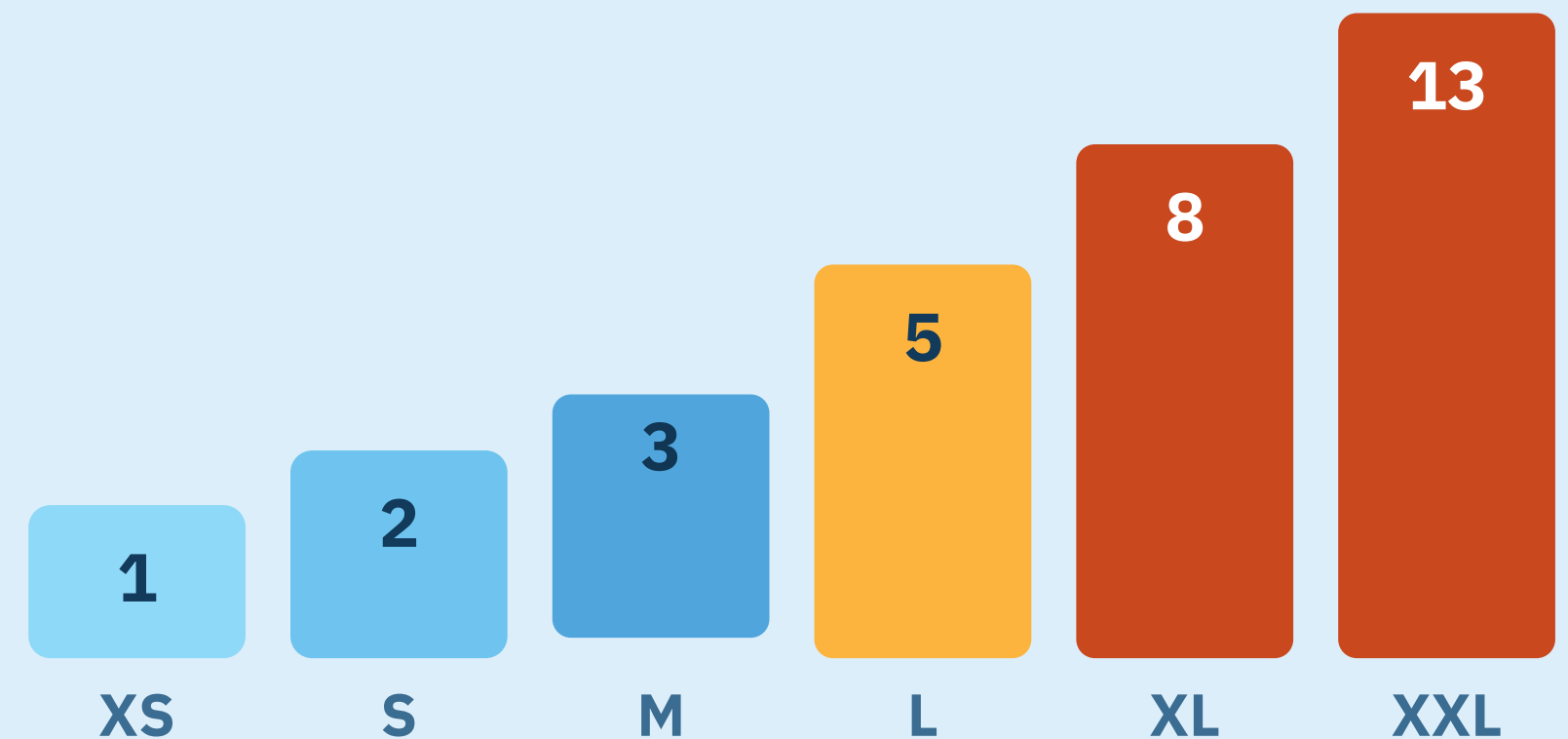
Three dimensions of a story point



T-shirt sizes and Fibonacci

- ◆ Story points acknowledge that humans are bad at estimating time-to-completion
- ◆ Instead, story points use relative T-Shirt sizes (S, M, L, XL)
- ◆ Most tools use Fibonacci numbers (1, 2, 3, 5, 8, 13, 21)

THE SCALE THE TEAM AGREES ON

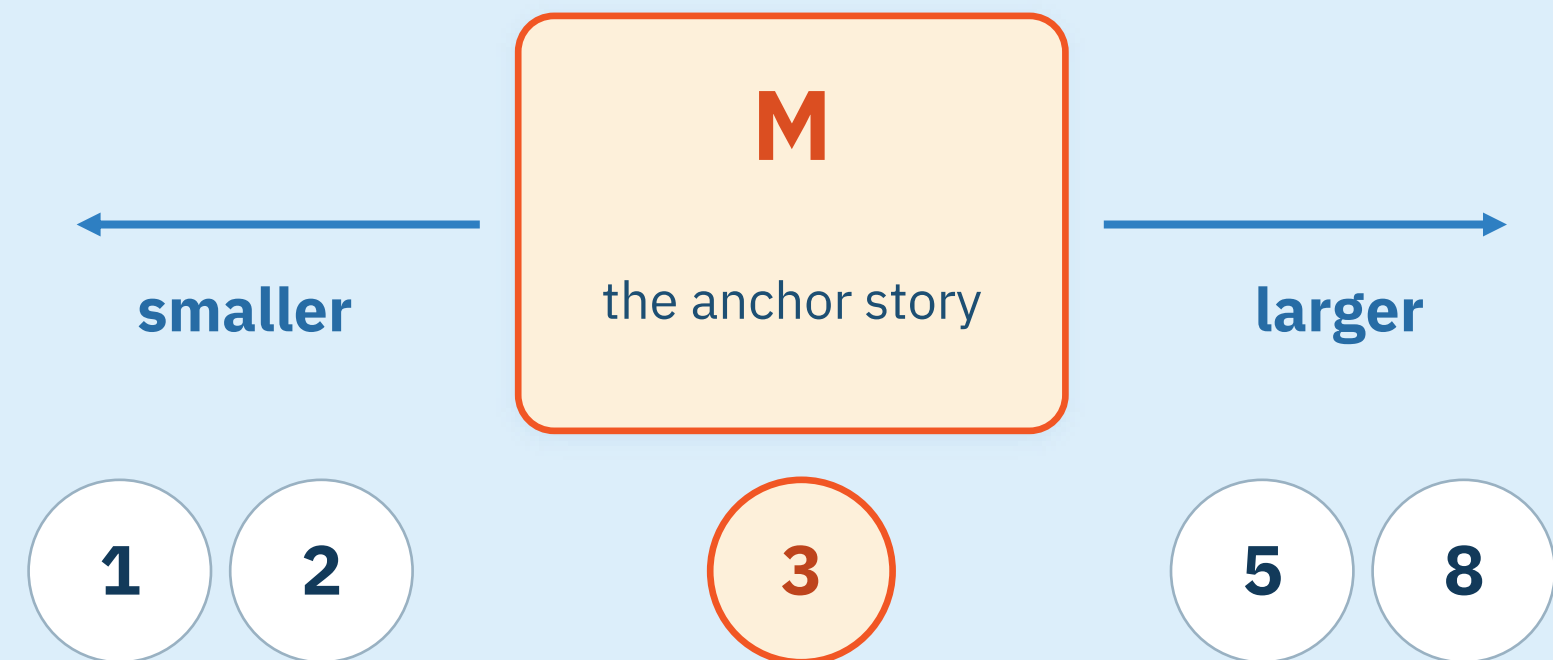


— RELATIVE ESTIMATION

The anchor story

- ◆ Since story points are relative, it's important to agree on what “medium” is
- ◆ Then evaluate from there
- ◆ Is it the same, larger, or smaller than medium?

EVERY ESTIMATE POINTS AT THE ANCHOR



Pick one story everybody has already built something like, call it medium, and write it on the board.

— STORY SIZE

- ◆ **A story should be small enough to be coded and tested within a single sprint iteration — ideally, just a few days**
- ◆ **Large stories should be broken down into smaller ones**

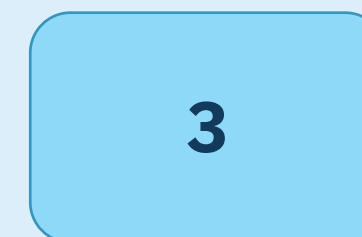
ONE SPRINT · TWO WEEKS



coded and tested inside the sprint



| sprint
| ends
|



split into
smaller ones

— STORY POINT ANTI-PATTERN

Points as hours

~~1 story point = 4 hours~~

**Equating a story point to wall-clock time.
Humans are bad at estimating wall-clock time.
Don't do it!**

WHAT HAPPENS IF YOU DO

- ◆ The number becomes a promise, and a promise gets argued about
- ◆ Two teams with different skills get compared on the same scale
- ◆ Velocity stops measuring capacity and starts measuring optimism

WHAT TO SAY INSTEAD

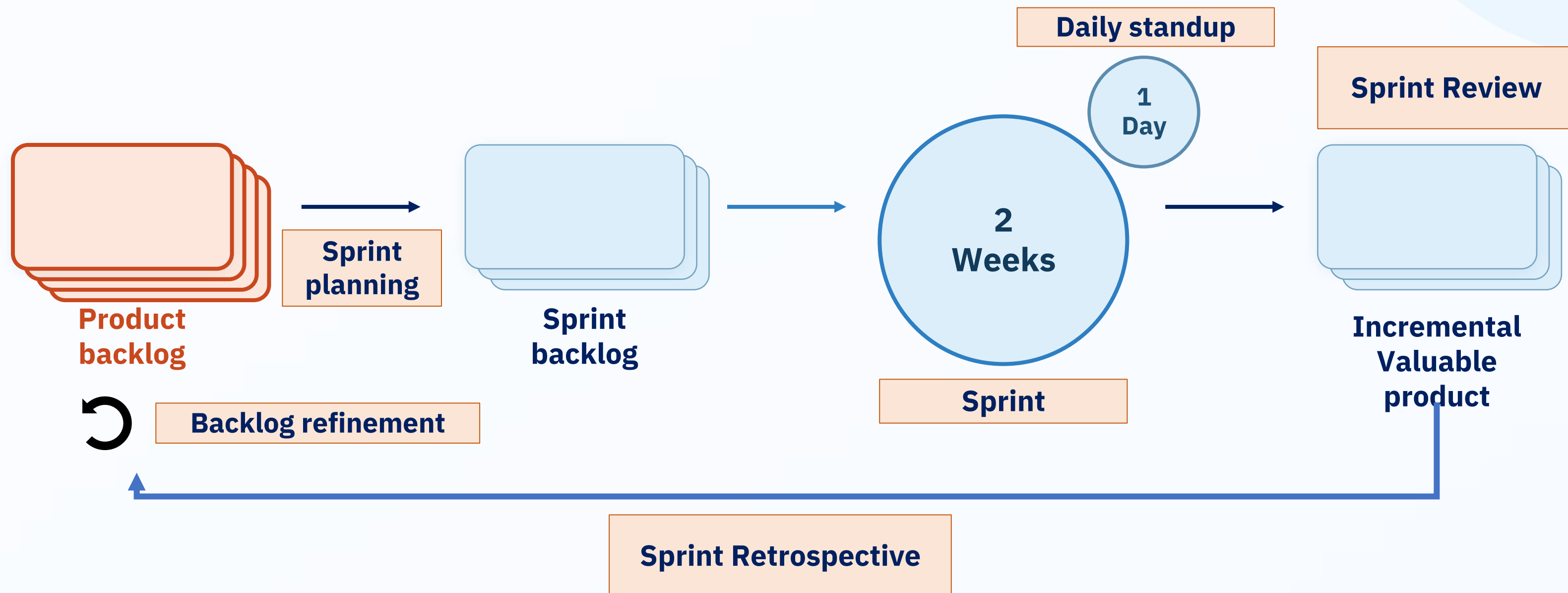
This story is about the same size as the one we called medium. Our team finished 18 points last sprint, so 18 is what we plan for — and the number of hours never enters the conversation.

 PART 08

The board

— STEPS IN THE SCRUM PROCESS

The product backlog



Contents and ranking

- ◆ A product backlog contains all the unimplemented stories not yet in a sprint
- ◆ Stories are ranked in order of importance and/or business value
- ◆ Stories are more detailed at the top, less detailed at the bottom

DETAIL FALLS DOWN THE LIST

top of the backlog

story, assumptions, acceptance criteria

middle

story and a rough size

bottom

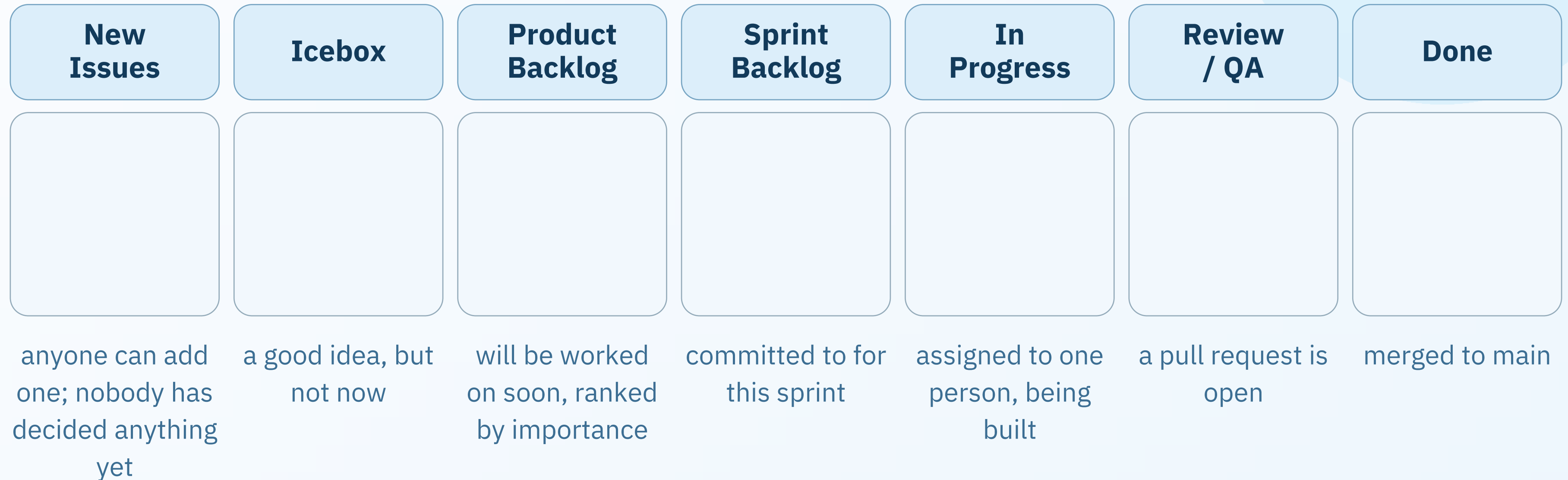
one line, no size yet

Four requirements

REQUIREMENTS

- ◆ **What: A service for counting things**
- ◆ **Must allow multiple counters**
- ◆ **Counters must persist across restarts of service**
- ◆ **Counters can be reset**

— THE BOARD



A CARD ONLY EVER MOVES TO THE RIGHT — EXCEPT BACK TO THE BACKLOG

— CREATING NEW STORIES

New Issues

THE FOUR REQUIREMENTS

- 1 What: A service for counting things
- 2 Must allow multiple counters
- 3 Counters must persist across restarts of service
- 4 Counters can be reset

— CREATING NEW STORIES

New Issues

Need a service for counting things

THE FOUR REQUIREMENTS

- 1 What: A service for counting things**
- 2** Must allow multiple counters
- 3** Counters must persist across restarts of service
- 4** Counters can be reset

THE REQUIREMENT

What: A service for counting things

As a User

I need a service that has a counter

So that I can keep track of how many times something was done

— CREATING THE NEXT STORY

New Issues

As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

#1

Must allow multiple counters

THE FOUR REQUIREMENTS

✓ What: A service for counting things

2 Must allow multiple counters

3 Counters must persist across restarts of service

4 Counters can be reset

THE REQUIREMENT

Must allow multiple counters

As a User

I need to have multiple counters

So that I can keep track of several counts at once

— CREATING THE NEXT STORY

New Issues

As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

#1

As a User, I need to have multiple counters, **So that** I can keep track of several counts at once

#2

THE FOUR REQUIREMENTS

✓ What: A service for counting things

✓ Must allow multiple counters

3 **Counters must persist across restarts of service**

4 Counters can be reset

THE REQUIREMENT

Counters must persist across restarts of service

As a Service Provider

I need the service to persist the last known count

So that users don't loose track of their counts after the service is restarted

— CREATING THE LAST STORY

New Issues

As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

#1

As a User, I need to have multiple counters, **So that** I can keep track of several counts at once

#2

As a Service Provider, I need the service to persist the last known count, **So that** users don't lose track of their counts after the service is restarted

#3

Counters can be reset

THE FOUR REQUIREMENTS

- ✓ What: A service for counting things
- ✓ Must allow multiple counters
- ✓ Counters must persist across restarts of service

4

Counters can be reset

THE REQUIREMENT

Counters can be reset

As a System Administrator

I need the ability to reset the counter

So that I can redo counting from the start

— STORIES IN THE BACKLOG

New Issues

As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

#1

As a User, I need to have multiple counters, **So that** I can keep track of several counts at once

#2

As a Service Provider, I need the service to persist the last known count, **So that** users don't lose track of their counts after the service is restarted

#3

As a System Administrator, I need the ability to reset the counter, **So that** I can redo counting from the start

#4

THE FOUR REQUIREMENTS

- ✓ What: A service for counting things
- ✓ Must allow multiple counters
- ✓ Counters must persist across restarts of service
- ✓ Counters can be reset

— PRIORITISING THE PRODUCT BACKLOG

New Issues	Icebox	Product Backlog
As a User, I need a service that has a counter, So that I can keep track of how many times something was done As a User, I need to have multiple counters, So that I can keep track of several counts at once As a Service Provider, I need the service to persist the last known count, So that users don't lose track of their counts after the service is restarted As a System Administrator, I need the ability to reset the counter, So that I can redo counting from the start		

— PRIORITISING THE PRODUCT BACKLOG

1

New Issues

As a User, I need to have multiple counters, **So that** I can keep track of several counts at once

As a Service Provider, I need the service to persist the last known count, **So that** users don't lose track of their counts after the service is restarted

As a System Administrator, I need the ability to reset the counter, **So that** I can redo counting from the start

Icebox

Product Backlog

As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

— PRIORITISING THE PRODUCT BACKLOG

2

New Issues

As a Service Provider, **I need** the service to persist the last known count, **So that** users don't lose track of their counts after the service is restarted

As a System Administrator, **I need** the ability to reset the counter, **So that** I can redo counting from the start

Icebox

As a User, **I need** to have multiple counters, **So that** I can keep track of several counts at once

Product Backlog

As a User, **I need** a service that has a counter, **So that** I can keep track of how many times something was done

— PRIORITISING THE PRODUCT BACKLOG

3

New Issues

As a System Administrator, **I need** the ability to reset the counter, **So that** I can redo counting from the start

Icebox

As a User, **I need** to have multiple counters, **So that** I can keep track of several counts at once

Product Backlog

As a User, **I need** a service that has a counter, **So that** I can keep track of how many times something was done

~~**As a** Service Provider, **I need**~~ the service to persist the last known count, **So that** users don't loose track of their counts after the service is restarted

— PRIORITISING THE PRODUCT BACKLOG

4

New Issues

Icebox

Product Backlog

As a User, I need to have multiple counters, **So that** I can keep track of several counts at once

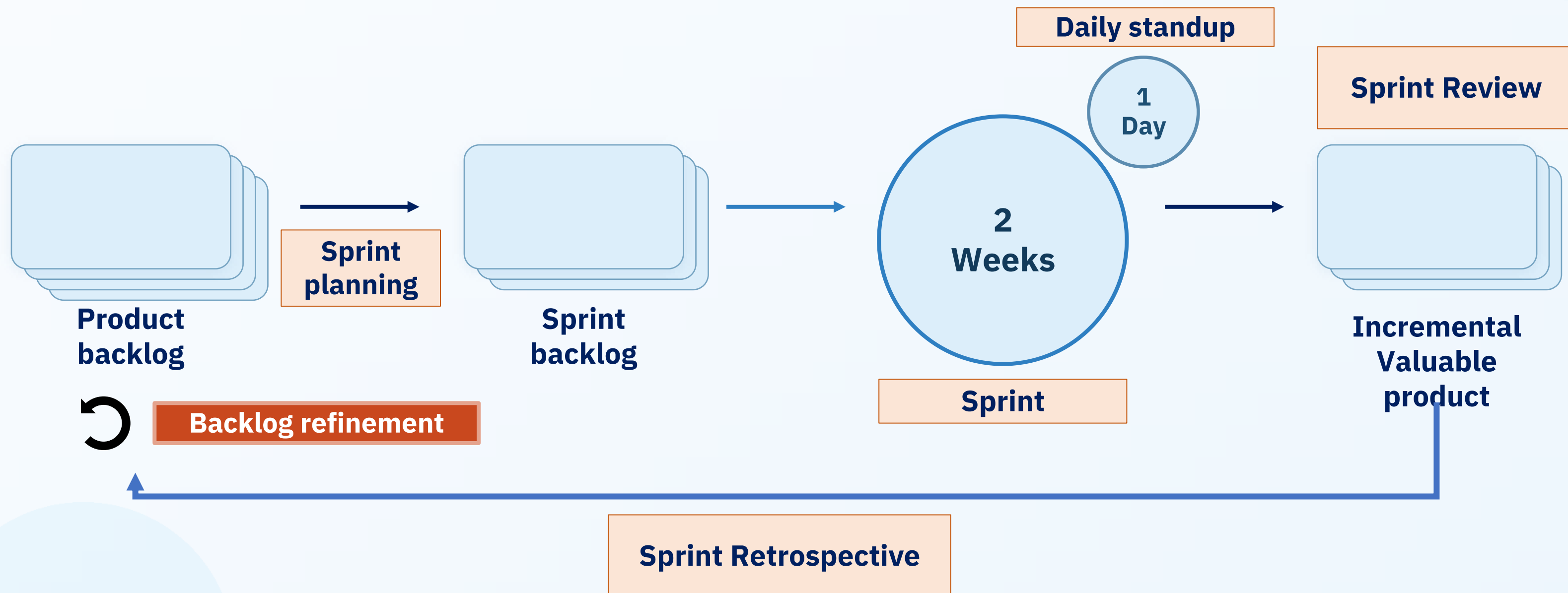
As a User, I need a service that has a counter, **So that** I can keep track of how many times something was done

As a Service Provider, I need the service to persist the last known count, **So that** users don't lose track of their counts after the service is restarted

As a System Administrator, I need the ability to reset the counter, **So that** I can redo counting from the start

— STEPS IN THE SCRUM PROCESS

Backlog refinement



Backlog grooming

- ◆ **Keep the product backlog ranked by priority so that the important stories are always on top**
- ◆ **Break large stories down into smaller ones**
- ◆ **Make sure that stories near the top of the backlog are groomed and complete**

— BACKLOG REFINEMENT MEETING

Attendees

WHO SHOULD ATTEND?

Product owner

Scrum master

Development team (optional)

Lead developer / architect

WHY EACH ONE IS THERE

Product owner

owns the order, so only this person can change it

Scrum master

Development team

— BACKLOG REFINEMENT MEETING

Goal of the meeting

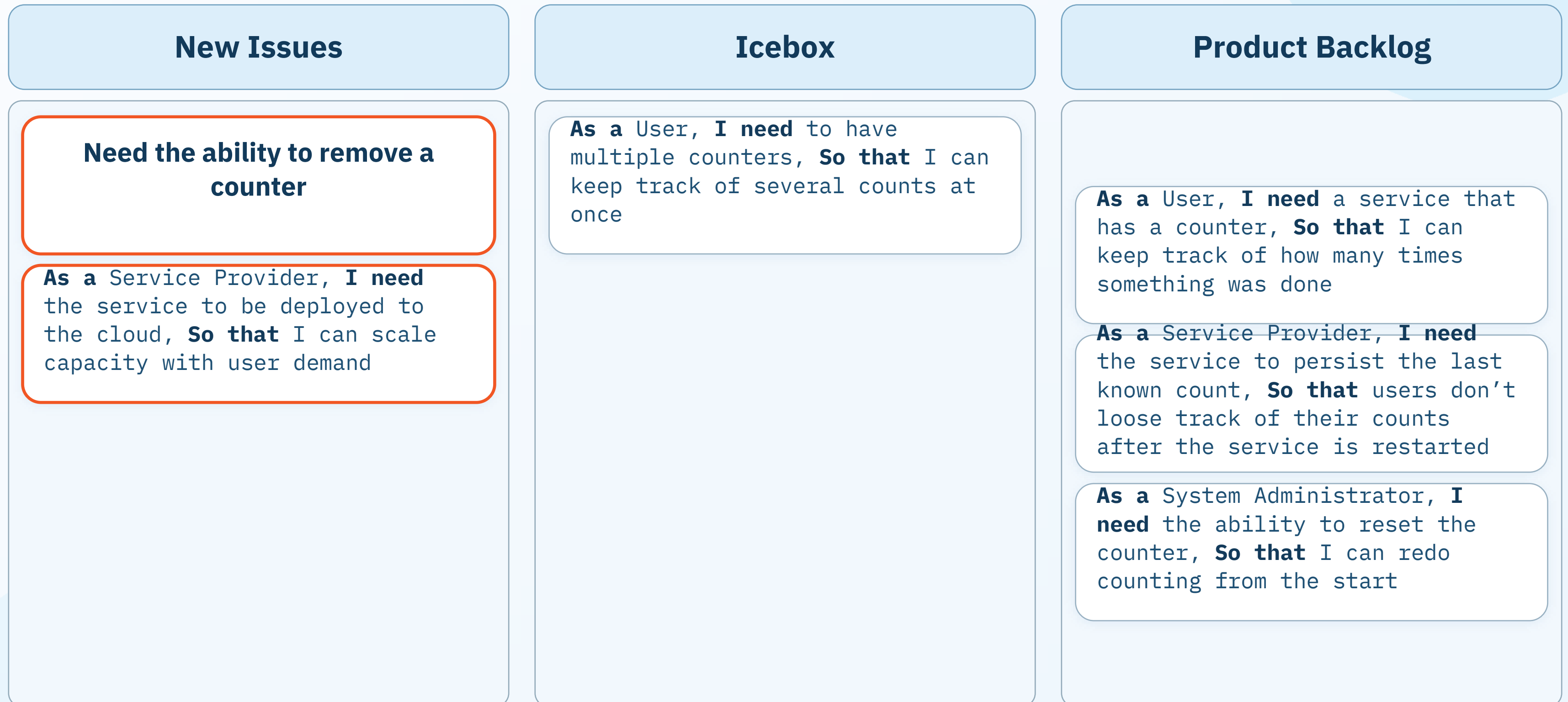
WHAT IS THE GOAL?

- ◆ Groom the backlog by ranking the stories in order of importance
- ◆ Make sure the story contains enough information for a developer to start working on it

THE TEST FOR “ENOUGH INFORMATION”

“Sprint ready” is the phrase to hold on to: a story a developer can pick up without asking anybody a question. If the first thing they have to do is find the product owner, the story was not ready.

— BACKLOG REFINEMENT WORKFLOW



— NEW ISSUE TRIAGE

- ◆ **Start with new issue triage**
- ◆ **Goal: At the end of backlog refinement the New Issues column is empty**

Three destinations

1

Move them into the product backlog

if they will be worked on soon

to the right, ranked

2

Move them into the icebox

if they are a good idea but not now

parked, not lost

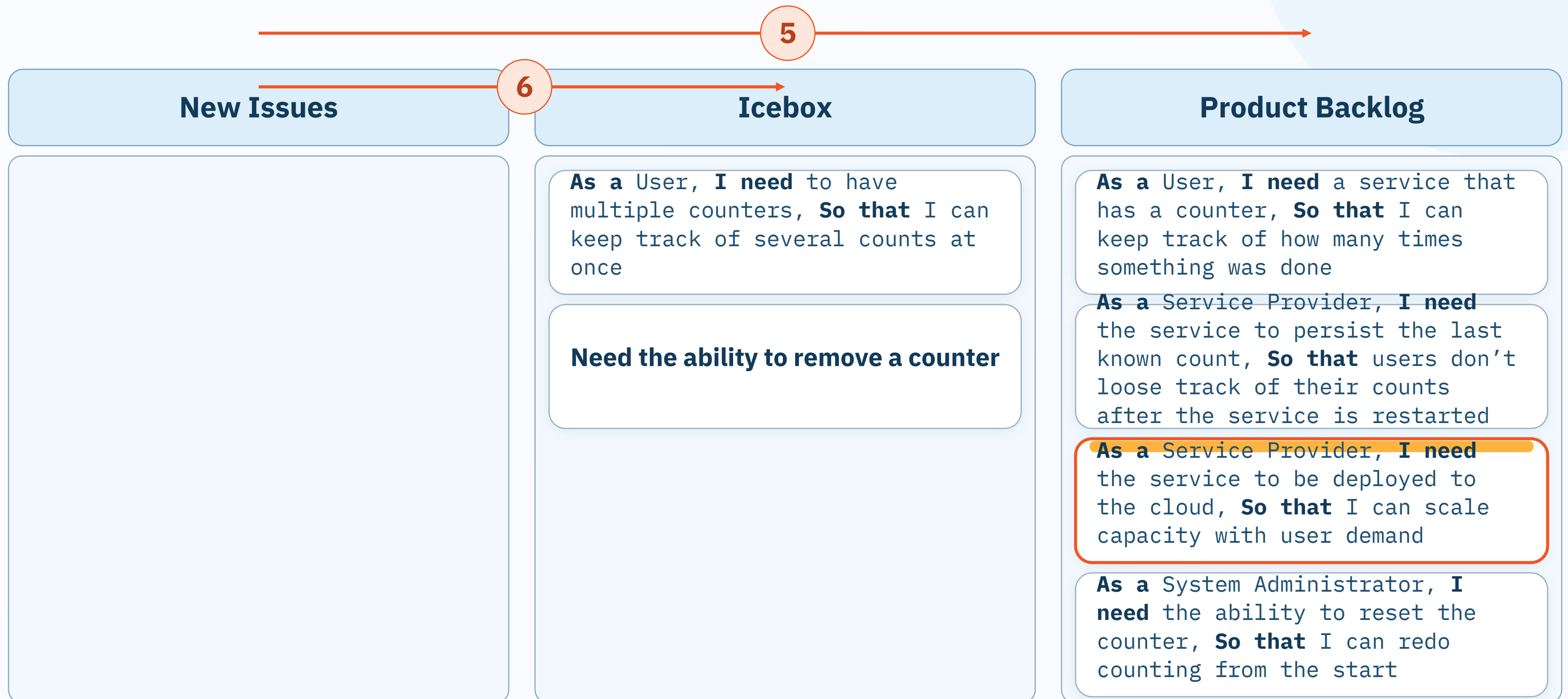
3

Reject them

if they are not where you want to go

closed, with a reason

NEW ISSUE TRIAGE



The four steps

1 Product owner sorts the product backlog in order of importance

2 The team may provide other technical information

3 Large vague items are split and clarified

4 Goal is to make the stories “sprint ready”

— THE STORY TEMPLATE

Definition of “Ready”

THE THREE BLOCKS

1

The story

who wants it, what they want, and why

2

Assumptions and Details

what the team already knows, so nobody has to guess

3

Acceptance Criteria

how anyone can tell it is finished

COPY THIS INTO THE ISSUE BODY

As a <some role>

I need <some function>

So that <some benefit>

Assumptions and Details:

- <anything you already know>

Acceptance Criteria:

Given <some precondition>

When <some event>

Then <some measurable outcome>

The five fields

As a User

I need a service that has a counter

So that I can keep track of how many times something was done

Assumptions and Details:

- Need a way to increment a counter
- Need a way to get the current value

Acceptance Criteria:

Given I have incremented the counter to 2

When I make a call to get the current value

Then it should return 2 as the counter value

Assignees

- No one

Labels

- None yet

Milestone

- No milestone

Estimate

- No estimate

— LABELS

Default GitHub labels

THE LABELS GITHUB GIVES YOU

bug

help wanted

duplicate

invalid

enhancement

question

**good first
issue**

wontfix

AND ONE YOU ADD YOURSELF

technical debt

— NEED A SERVICE THAT HAS A COUNTER

As a User

I need a service that has a counter

So that I can keep track of how many times something was done

Assumptions and Details:

- Need a way to increment a counter
- Need a way to get the current value

Acceptance Criteria:

Given I have incremented the counter to 2

When I make a call to get the current value

Then it should return 2 as the counter value

Assignees

- No one

Labels

Enhancement

Milestone

- No milestone

Estimate

- No estimate

— MUST PERSIST COUNTER ACROSS RESTARTS

As a Service Provider

I need the service to persist the last known count

So that users don't lose track of their counts after the service is restarted

Assumptions and Details:

- We will use a Redis database
- Counter will be stored as name value pair

Acceptance Criteria:

Given I have incremented the counter to 2

When I restart the service

Then the counter should still return 2

Assignees

- No one

Labels

Enhancement

Milestone

- No milestone

Estimate

- No estimate

— DEPLOY SERVICE TO THE CLOUD

As a Service Provider

I need the service to be deployed to the cloud

So that I can scale capacity with user demand

Assumptions and Details:

- We will use a managed cloud platform
- Deploy as a container image

Acceptance Criteria:

Given I have deployed to the cloud

When a customer comes to our URL

Then our service will be available

Assignees

- No one

Labels

Technical debt

Milestone

- No milestone

Estimate

- No estimate

Definition of technical debt

- ◆ **Technical debt is anything you need to do that doesn't involve creating a new feature**
- ◆ **Technical debt builds up when you take shortcuts but may also occur naturally**

THE TWO WAYS IT ARRIVES

A shortcut

taken on purpose, to ship this sprint

Time passing

a library goes out of support; the database version ends

Four examples

1 Code refactoring

2 Setup and maintenance of environments

the pipeline, the test database, the staging server

3 Changing technology like databases

a migration no user will ever notice

4 Updating vulnerable libraries

the security advisory

— ABILITY TO RESET THE COUNTER

As a System Administrator

I need the ability to reset the counter

So that I can start counting something new

Assumptions and Details:

- Use the action `POST /counters/{name}/reset`

Acceptance Criteria:

Given the counter has advanced to 5

When I call reset on the service

And I request the current counter

Then I should see 0 returned from the service

Assignees

- No one

Labels

Enhancement

Milestone

- No milestone

Estimate

- No estimate

— BACKLOG REFINEMENT TIPS

Three habits

1

You should refine the backlog every sprint to ensure the priorities are correct

2

Have at least two sprints' worth of stories groomed

3

The more time you spend refining the backlog, the easier sprint planning will be

TWO SPRINTS AHEAD

Sprint 1 groomed

Sprint 2 groomed

Sprint 3 one line each

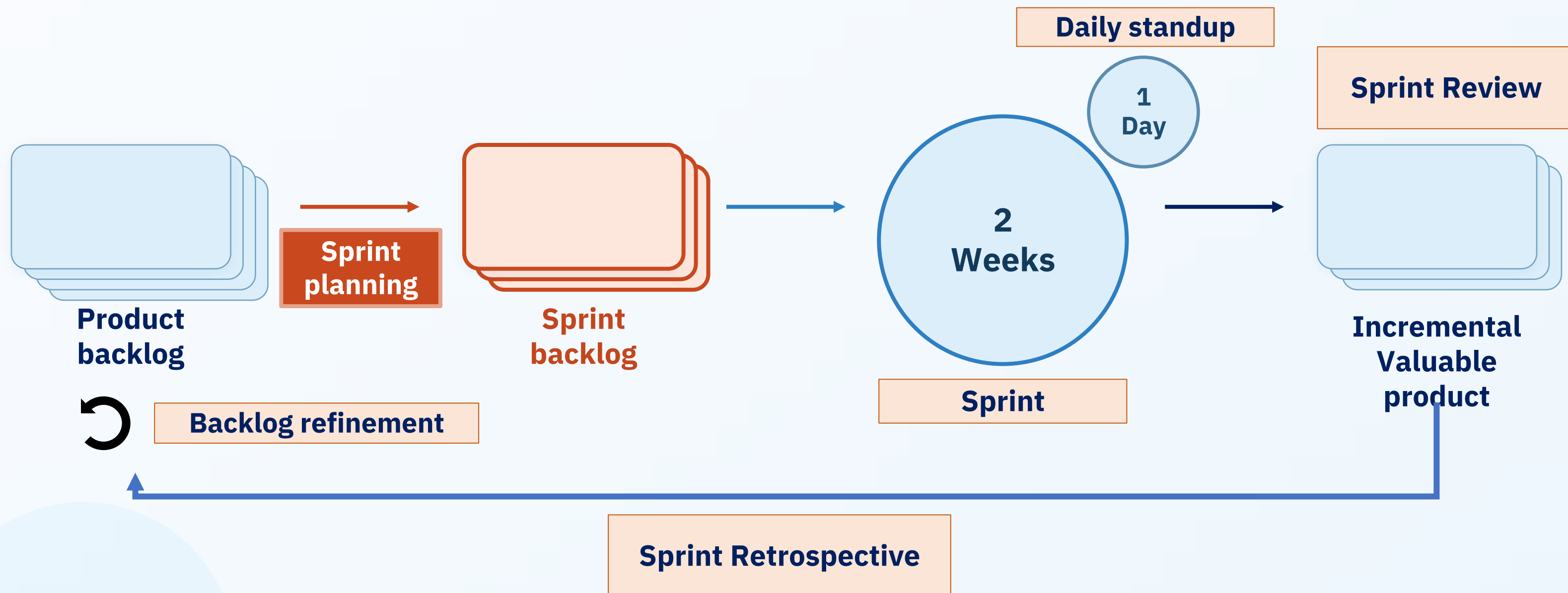
Sprint 4 one line each

 PART 09

Sprint planning and the sprint

— STEPS IN THE SCRUM PROCESS

Sprint planning



Purpose of the meeting

- ◆ **The purpose of sprint planning is to define what can be delivered in the sprint and how that work will be achieved**
- ◆ **This is accomplished by producing a sprint backlog**

Attendees

Product owner

brings the ranked backlog and the sprint goal and **KEEP QUIET UNLESS SPOKEN TO**

Scrum master

runs the meeting and keeps it in its time box

Development team

decides how much fits, and says so out loud

— SPRINT PLANNING GOALS

The sprint goal

- ◆ **Each sprint should have a clearly defined business goal**
- ◆ **The product owner describes the goal and product backlog items supporting it**
- ◆ **It's important for the whole team to understand why they are building the increment**

Four steps for the team

1 Takes stories from the top of the product backlog and assigns them to the sprint backlog

2 Assigns story points and labels

3 Ensures each story contains enough information for a developer to start working on it

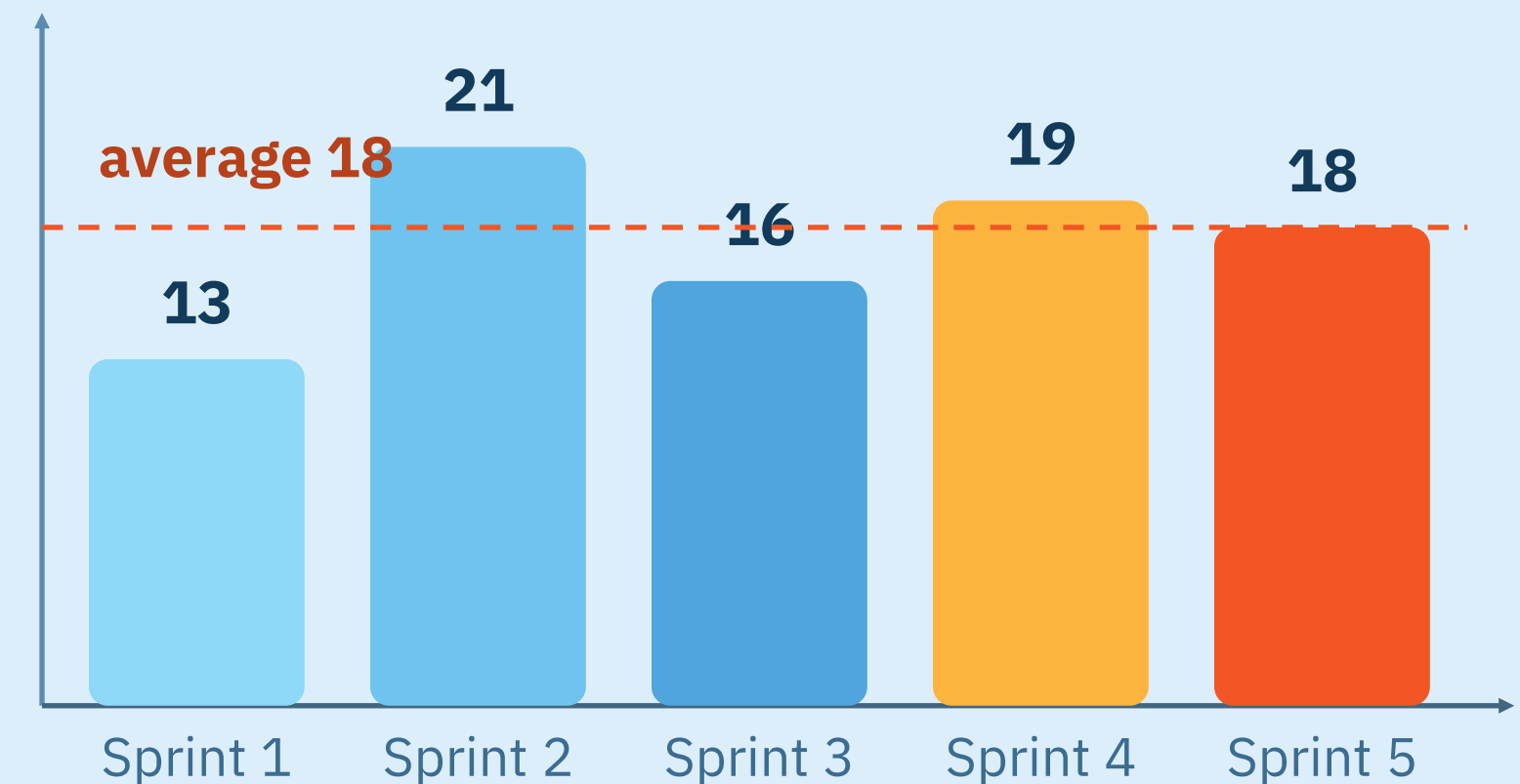
4 Stops adding stories when the team's velocity is reached

— TEAM VELOCITY

Definition of velocity

- ◆ The number of story points a team can complete in a single sprint
- ◆ This will change over time as the team gets better at estimating and better at executing
- ◆ The velocity is unique to the team because the story point assignment is unique to the team

FIVE SPRINTS OF ONE TEAM



— THE SPRINT MILESTONE

- ◆ **Create a sprint milestone to start the sprint**
- ◆ **The milestone title should be short**
- ◆ **The description should document the milestone goal**
- ◆ **The duration should be 2 weeks**

— THE SPRINT MILESTONE

The milestone form

CREATE A NEW MILESTONE

Estimate issues and pull requests to see how a milestone is progressing in the burndown chart.

Milestone title

Sprint 1: Single Counter

Description

The goal is to get a single counter working and deployed to the cloud

Start and due date

8 Sep 2026 → 19 Sep 2026

Create milestone

WHAT EACH BOX IS FOR

Title

short enough to read on a card: “Sprint 1: Single Counter”

Description

this is where the sprint goal is describe

Dates

two weeks — they are what the burndown’s horizontal axis is made of

— SPRINT PLANNING

0 Story Points

New Issues	Icebox	Product Backlog	Sprint Backlog	In Progress	Review / QA	Done
	Multiple counters #2 Remove a counter #6	Single counter #1 Persist the count #3 Deploy to the cloud #5 Reset the counter #4				

— NEED A SERVICE THAT HAS A COUNTER

As a User

I need a service that has a counter

So that I can keep track of how many times something was done

Assumptions and Details:

- Need a way to increment a counter
- Need a way to get the current value

Acceptance Criteria:

Given I have incremented the counter to 2

When I call the counter service

Then it should return 2 as the counter value

Assignees

- No one

Labels

Enhancement

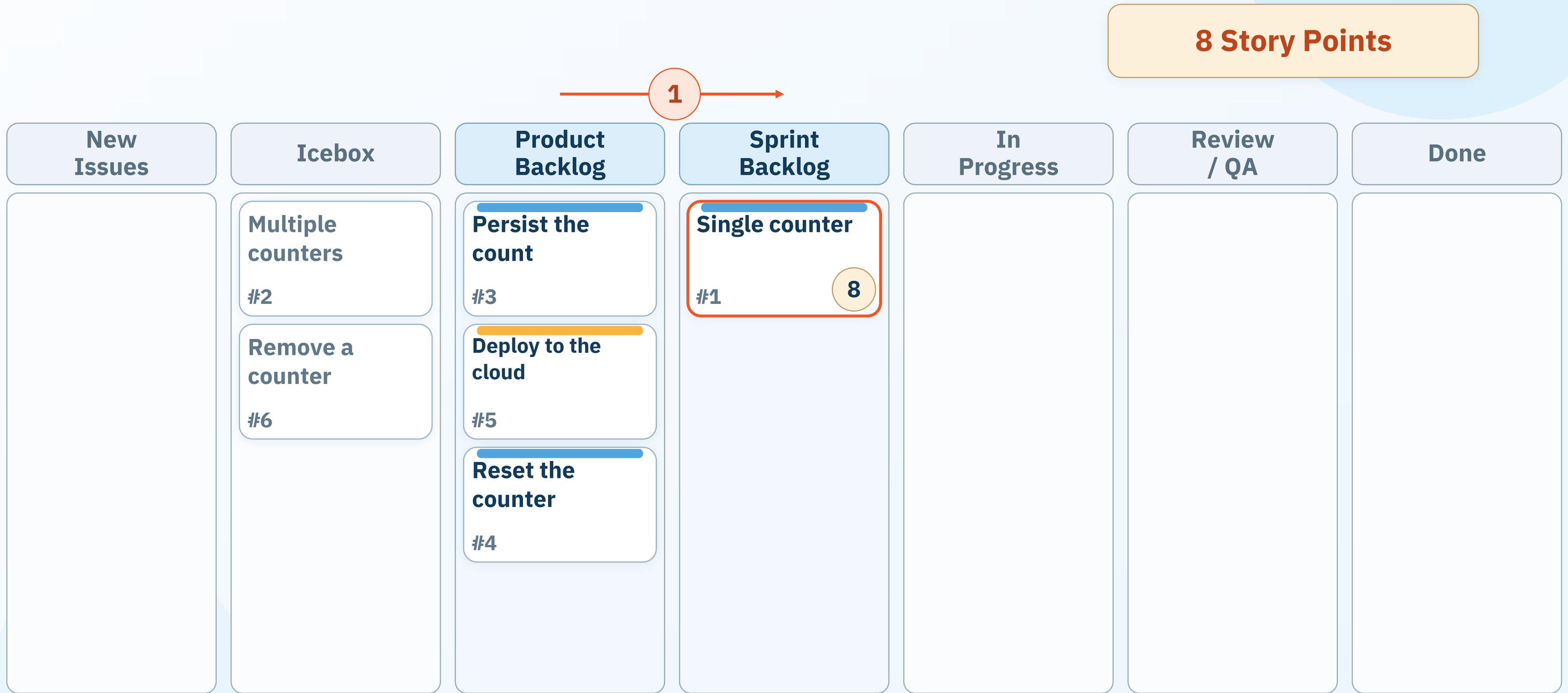
Milestone

- Sprint 1: Single Counter

Estimate

- 8 - Large

— SPRINT PLANNING



— MUST PERSIST COUNTER ACROSS RESTARTS

As a Service Provider

I need the service to persist the last known count

So that users don't lose track of their counts after the service is restarted

Assumptions and Details:

- We will use a Redis database
- Counter will be stored as name value pair

Acceptance Criteria:

Given I have incremented the counter to 2

When I restart the service

Then the counter should still return 2

Assignees

- No one

Labels

Enhancement

Milestone

- Sprint 1: Single Counter

Estimate

- 5 - Medium

— DEPLOY SERVICE TO THE CLOUD

As a Service Provider

I need the service to be deployed to the cloud

So that I can scale capacity with user demand

Assumptions and Details:

- We will use a managed cloud platform
- Deploy as a container image

Acceptance Criteria:

Given I have deployed to the cloud

When a customer comes to our URL

Then our service will be available

Assignees

- No one

Labels

Technical debt

Milestone

- Sprint 1: Single Counter

Estimate

- 5 - Medium

— SPRINT PLANNING

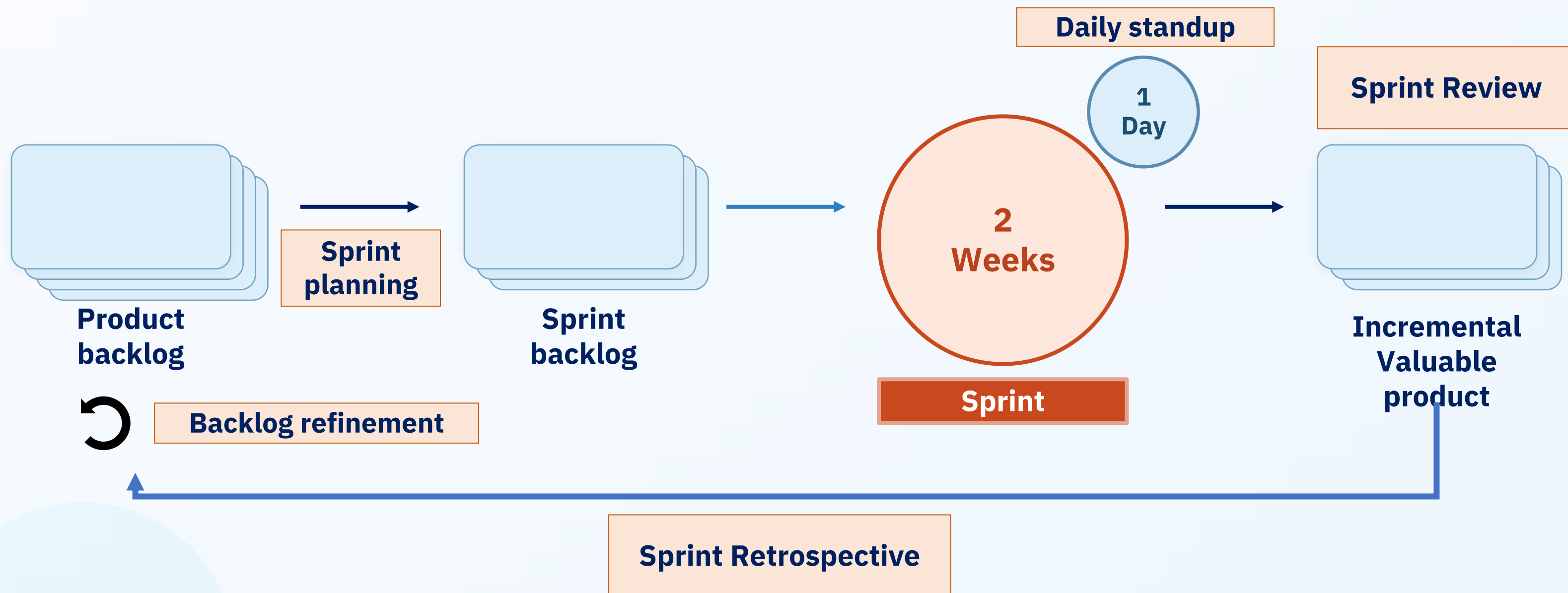
18 Story Points



New Issues	Icebox	Product Backlog	Sprint Backlog	In Progress	Review / QA	Done
	Multiple counters #2 Remove a counter #6	Reset the counter #4	Single counter #18 Persist the count #35 Deploy to the cloud #55			

— STEPS IN THE SCRUM PROCESS

The sprint



— THE SPRINT

- ◆ **A sprint is one iteration through the design, code, test, deploy cycle**
- ◆ **It is usually 2 weeks in duration**
- ◆ **Every sprint should have a goal**

— DAILY EXECUTION

Three steps each day

EACH DAY

1 Take the next highest priority item from the sprint backlog



2 Assign it to yourself



3 Move it in process

WHAT “NEXT HIGHEST” MEANS

the top unassigned card in the Sprint Backlog column

not the one you find most interesting

not the one nearest your own code

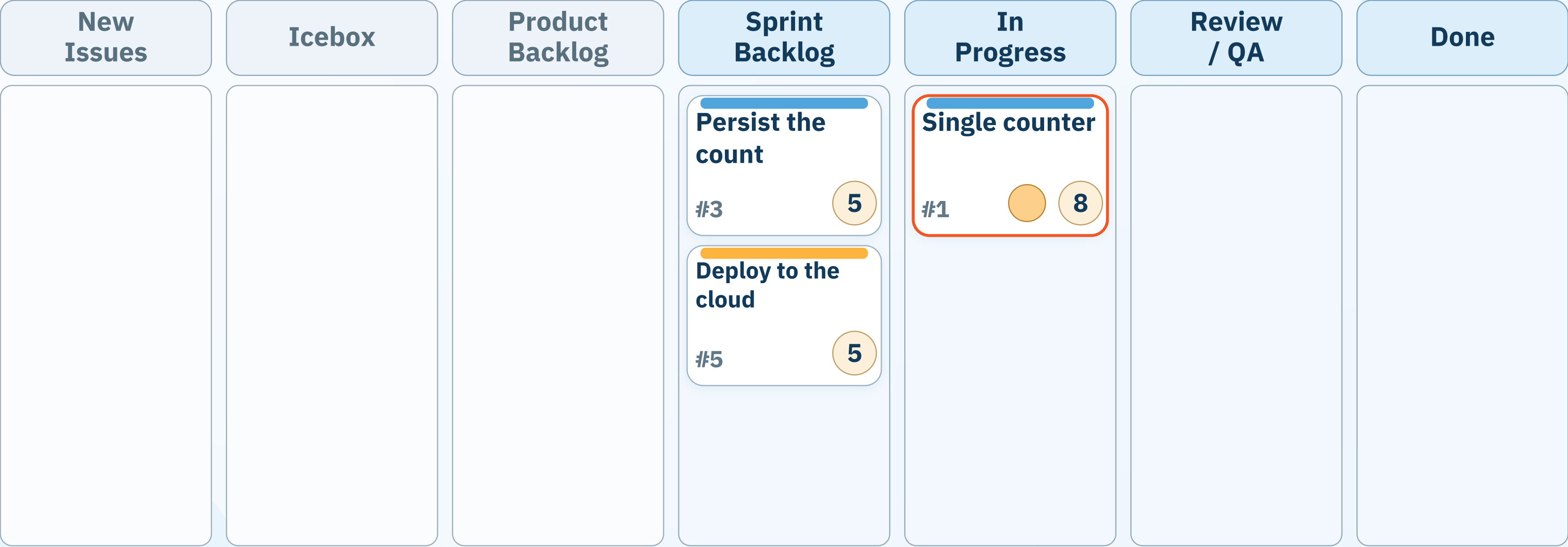
not the one you started thinking about yesterday

— THE SPRINT BACKLOG

New Issues	Icebox	Product Backlog	Sprint Backlog	In Progress	Review / QA	Done
			Single counter #18 Persist the count #35 Deploy to the cloud #55			

WIP 1

— IN PROGRESS



WIP 1

— NEED A SERVICE THAT HAS A COUNTER

As a User

I need a service that has a counter

So that I can keep track of how many times something was done

Assumptions and Details:

- Need a way to increment a counter
- Need a way to get the current value

Acceptance Criteria:

Given I have incremented the counter to 2

When I call the counter service

Then it should return 2 as the counter value

Assignees

- the developer who took it

Labels

Enhancement

Milestone

- Sprint 1: Single Counter

Estimate

- 8 - Large

Three rules for moving cards

1

No one should have more than one story assigned to them unless they are blocked

WIP 1

2

When you are finished, move the story to Review / QA and open a pull request

In Progress → Review / QA

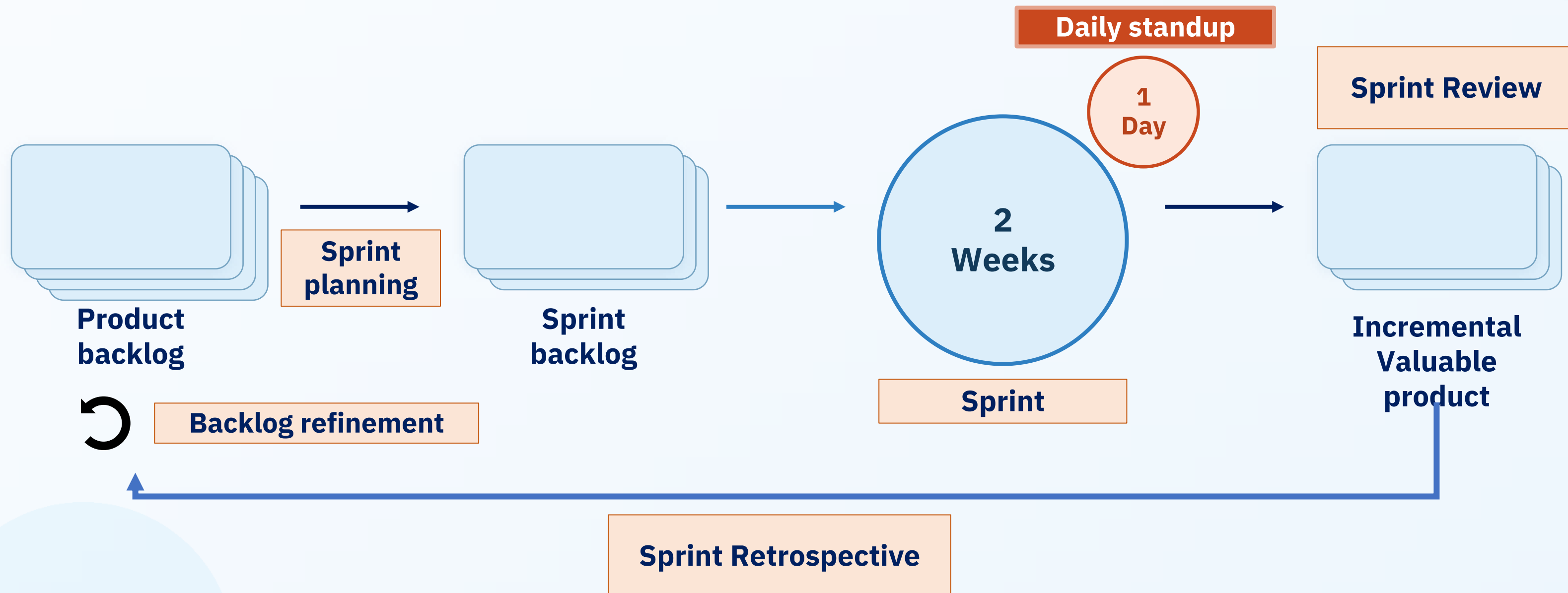
3

When the pull request is merged, move the story to the Done column

Review / QA → Done

— STEPS IN THE SCRUM PROCESS

The daily stand-up



— DAILY STAND-UP

- ◆ **Occurs every day at the same time and place**
- ◆ **Sometimes called the “daily Scrum”**
- ◆ **Each team member briefly reports on their work**

— DAILY STAND-UP

The time box

Called a “stand-up” because everyone should remain standing during the meeting to keep it short.

15 minutes

the whole meeting, for the whole team

Not a status report

the team is talking to each other, not to a manager

Attendees

Scrum master

listens for the block and try to solve it

Development team

Product owner (optional)

welcome, but this is not their meeting

— THE THREE QUESTIONS

1

What did I accomplish the previous day?

2

What will I work on today?

3

What blockers or impediments are in my way?

— IMPEDIMENTS AND BLOCKERS

The exception to WIP 1

- ◆ Impediments identified by the team should be unblocked by the scrum master
- ◆ Developers that are blocked should work on the next story

THE ONE EXCEPTION TO WIP 1

#1 blocked



#2 started

Two cards in one person's name is allowed only while one of them cannot move.

— TABLED TOPICS

After the stand-up

- ◆ **Topics raised during the daily stand-up should be held until the meeting has ended**
- ◆ **Anyone interested in those topics can stay to discuss**

00:00 – 00:15

THE STAND-UP

three answers each, no discussion

00:15 – ...

WHOEVER STAYS

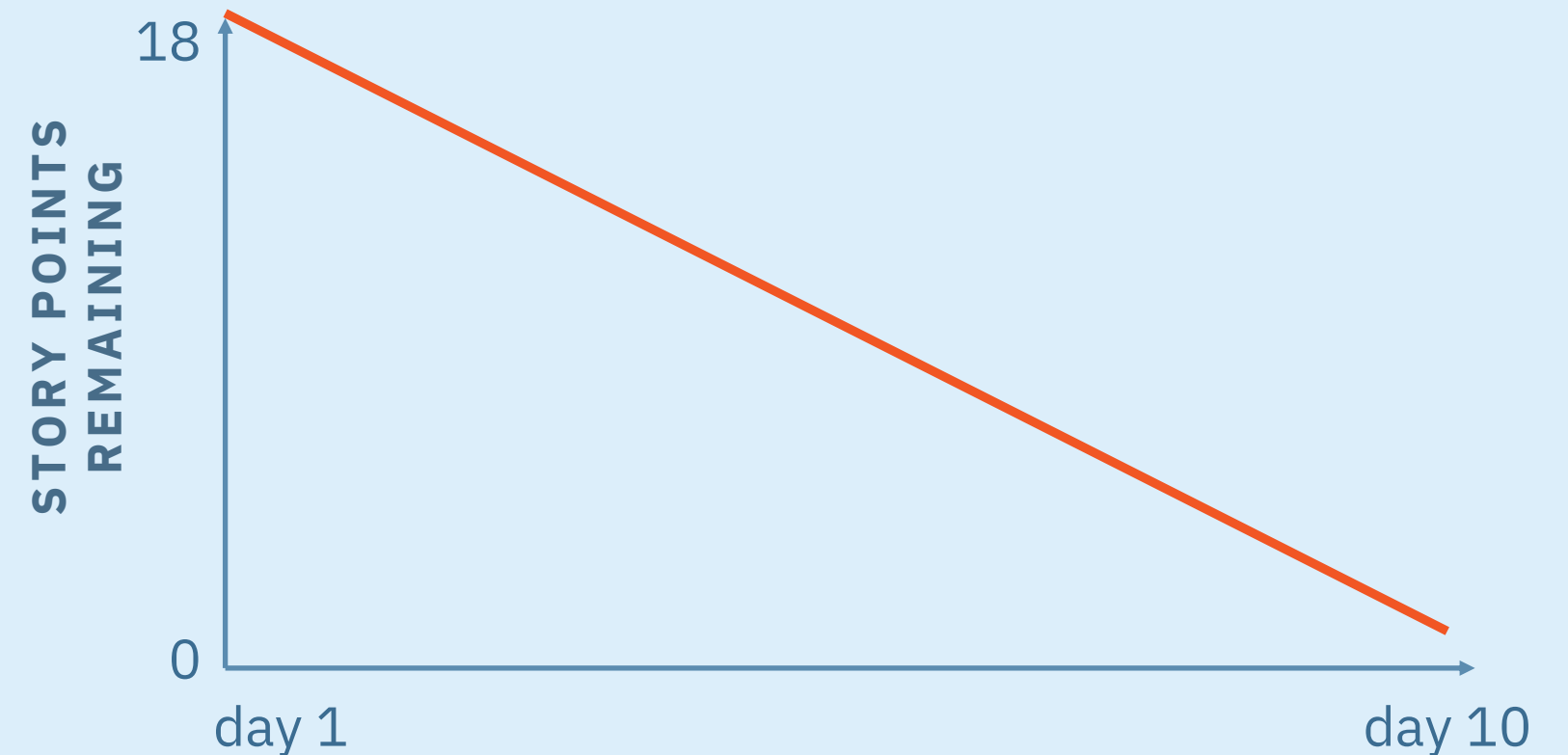
the tabled topics, with the two or three people they concern

— BURNDOWN CHART

Definition and axes

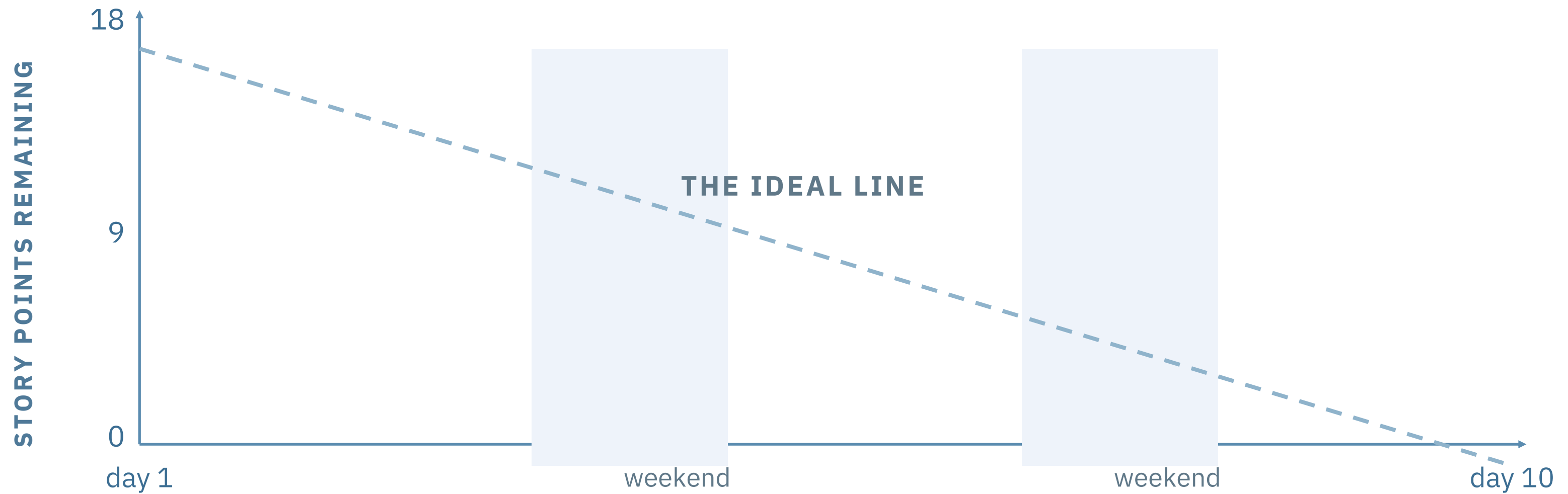
- ◆ **The measurement of story points completed vs story points remaining for a sprint**
- ◆ **Over time the story points remaining should go down, hence the name: burndown**

THE AXES



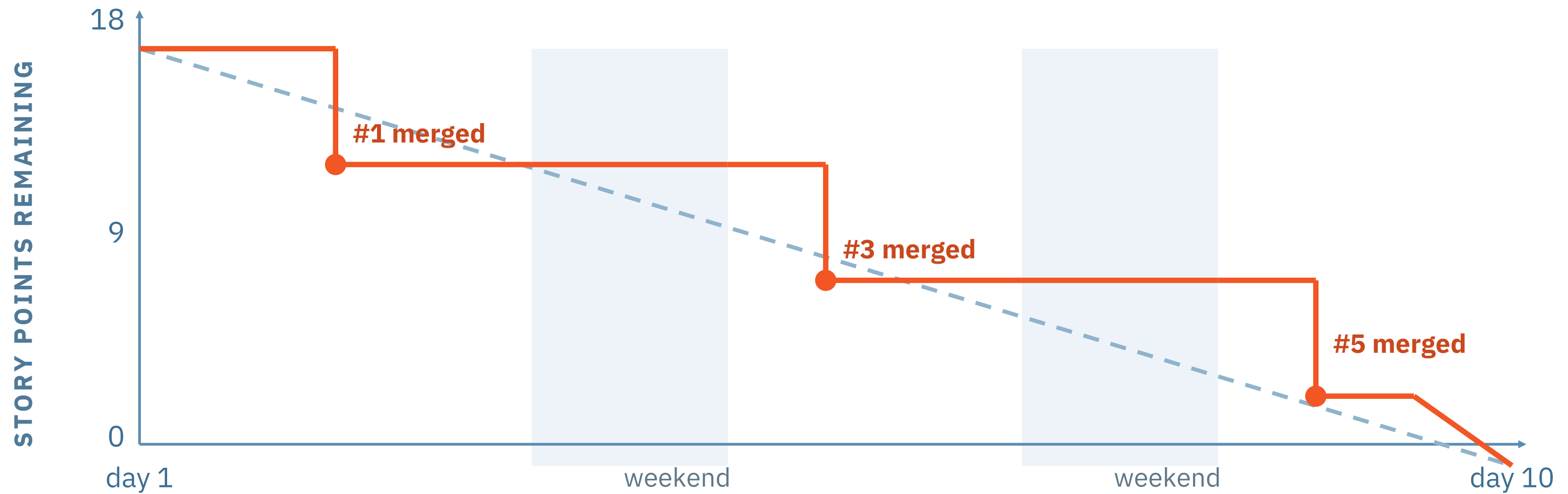
— BURNDOWN CHART EXAMPLE

The ideal line



BURNDOWN CHART EXAMPLE

The actual line

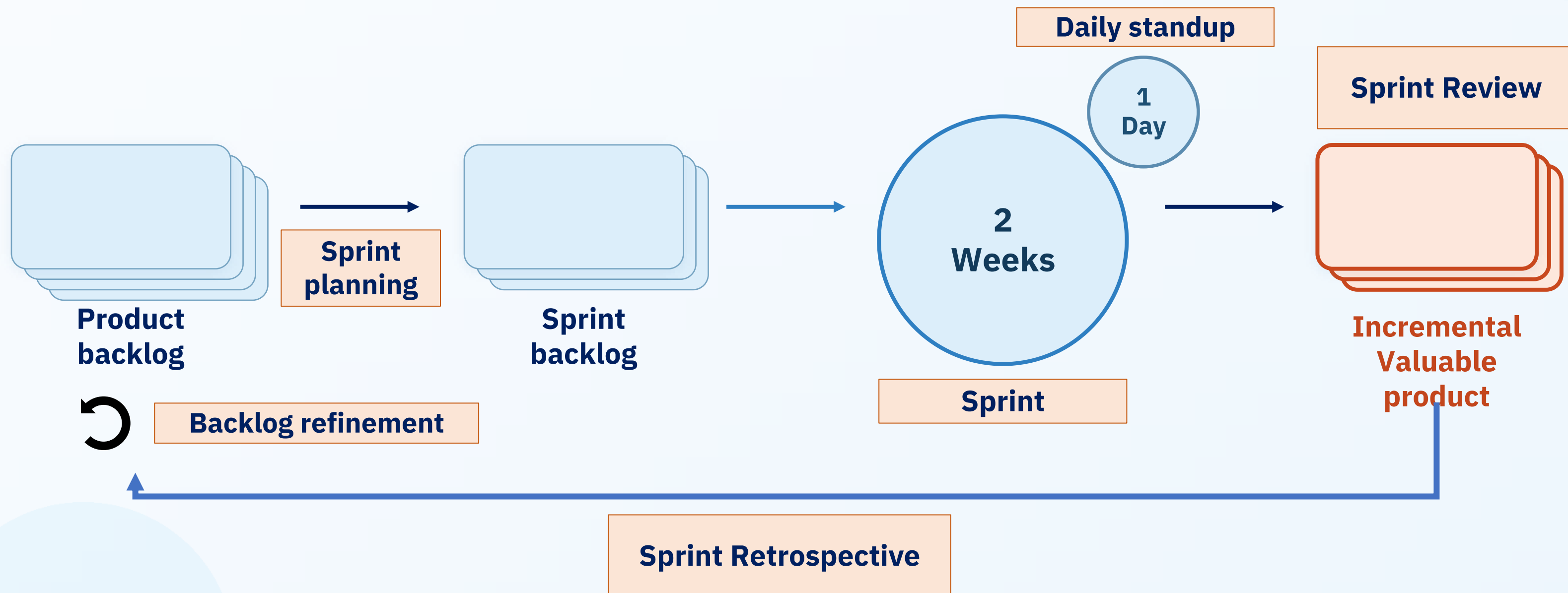


 PART 10

Review, retrospective and metrics

— STEPS IN THE SCRUM PROCESS

The valuable product



The demonstration

- ◆ **Live demonstration of implemented stories**
- ◆ **Product owner determines if stories are done based on acceptance criteria**
- ◆ **Done stories are closed**

Attendees

Product owner

decides done or not
done

Scrum master

runs the session

Development team

demonstrates the work

Stakeholders

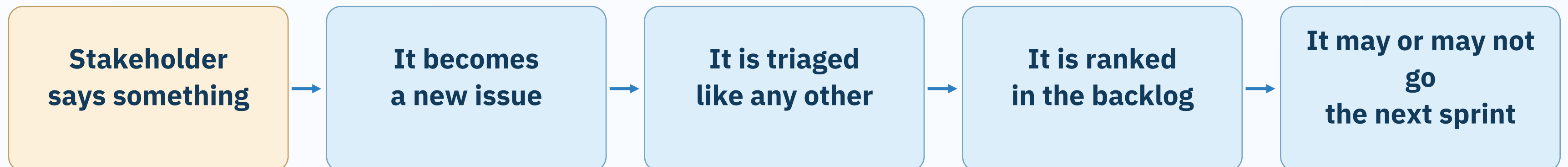
the people who asked
for it

Customers (optional)

welcome, when there
are any to invite

The feedback path

- ◆ **Feedback gets converted into new product backlog stories**
- ◆ **This is where iterative development allows the creation of products that couldn't have been specified up-front in a plan-driven approach**



— REJECTED STORIES

Handling a rejected story

1 Add a label to indicate this and close them

2 Write a new story with new acceptance criteria

3 This will keep the velocity more accurate

WHY NOT JUST CARRY IT OVER

A story that is half-finished and moved to the next sprint gets counted twice, or not at all.

Closing it and writing a new story for the remaining work keeps both sprints' numbers honest — and keeps the credit with the person who did the work.

— SPRINT RETROSPECTIVE

Purpose of the retrospective

- ◆ **A meeting to reflect on the sprint**
- ◆ **Measures the health of the process**
- ◆ **The development team must feel comfortable to speak freely**

Attendees

Scrum master

facilitates, and owns the change that comes out of it

Development team

everyone who did the work, and nobody else

The three questions

1

What went well?

(keep doing)

2

What did not go well?

(stop doing)

3

**What should we
change for the next
sprint?**

(start doing)

— SPRINT RETROSPECTIVE

Acting on the outcome

- ◆ **This is critical for maintaining a healthy team**
- ◆ **The scrum master must ensure that changes are made as a result of the feedback**
- ◆ **The goal is to improve for the next sprint**

— METRICS

Two habits

You can't improve what you can't measure.

1 High performing teams use metrics to continually improve

2 They take baselines and set goals and measure against them

— METRICS

Vanity metrics

We had 10,000 “hits” to the website!!!

What does that mean?

- ◆ **Do you really know what actions you took in the past that drove those visitors to you?**
- ◆ **Do you really know which actions to take next?**

Actionable metrics

1 Use A/B split-test to deploy new features to 50% of users

2 Measure behavior of those who saw the feature and those who did not

3 Do more of what produced the desired behavior

THE SPLIT

50%

saw the new feature

50%

did not

The baseline

BASELINE

- ◆ **It currently requires six team members 10 hours to deploy a new release of your product**
- ◆ **This costs you \$X for every release**

The goal

GOAL

- ◆ **Reduce deployment time from 10 hours to 2 hours**
- ◆ **Increase percentage of defects detected in testing from 25% to 50%**

— TOP FOUR ACTIONABLE METRICS

The four metrics

1

Mean Lead Time

How long does it take from idea to production?

speed

2

Release Frequency

How often can you deliver changes?

speed

3

Change Failure Rate

How often do changes fail?

stability

4

Mean Time to Recovery (MTTR)

How quickly can you recover from failure?

stability

— EXAMPLE METRICS

Five examples

- ◆ **Reduce time-to-market for new features**
- ◆ **Increase overall availability of the product**
- ◆ **Reduce the time it takes to deploy a software release**
- ◆ **Increase the percentage of defects detected in testing before production release**
- ◆ **Provide performance and user feedback to the team in a more timely manner**

— END OF SPRINT ACTIVITIES

Four activities

1 Move stories from done to closed

2 Close the current milestone

3 Create a new sprint milestone

4 Adjust unfinished work

— UNTOUCHED STORIES

Where the story goes

- ◆ Stories not worked on can be moved to the top of the product backlog
- ◆ Resist the urge to move them to the next sprint
- ◆ Remember to un-assign them from the sprint milestone

WHERE IT GOES

Sprint Backlog



Product Backlog, top



Next sprint

— UNFINISHED STORIES

Credit and velocity

Do not move unfinished stories into the next sprint!

◆ **Give the developer credit for the work they did**

◆ **This will keep your velocity more accurate**

— UNFINISHED STORIES

Three steps

1 Adjust the description and story points of the unfinished story, label it unfinished, and move it to done

2 Write a new story for the remaining work

3 Assign remaining story points and move it to the next sprint

THE ARITHMETIC

8 estimated

5 done this sprint

3 the new story

— READY FOR THE NEXT SPRINT

The four checks



All stories assigned to the current sprint are closed



All unfinished stories are reassigned



The sprint milestone is closed



A new sprint milestone is created

Six anti-patterns



No real product owner

somebody has the title, nobody has the authority



Teams are too large

more than nine and the conversation stops working



Teams are not dedicated

everyone is on three projects at once



Teams are too geographically distributed

no overlapping hours, no conversation



Teams are siloed

one skill per team, so nothing finishes end to end



Teams are not self-managing

they wait to be told what to do next

Agile Anti-Patterns

...and you should not wonder why.

Structure



The accountabilities of product owner, development team(s) and Scrum master are identified and enacted



Work is organized in consecutive sprints of 2–4 weeks or fewer



There is an ordered product backlog



There is a sprint backlog with a visualization of remaining work for the sprint

Events



At sprint planning a forecast, a sprint backlog, and a sprint goal are created



The result of the daily Scrum is work being re-planned for the next day



No later than by the end of the sprint, a Done increment is created

Feedback loops



Stakeholders offer feedback as a result of inspecting the increment at the sprint review



Product backlog is updated as a result of the sprint review



Product owner, development team(s) and Scrum master align on the work process for their next sprint at the sprint retrospective

Questions?

Next week: architecture, APIs, and turning acceptance criteria into tests.

THIS AFTERNOON

**Board, backlog, Sprint 4
milestone**

BEFORE NEXT WEEK

One sprint-ready story each

